

Why Functional Programming Matters



John Hughes & Mary Sheeran

QuviQ

CHALMERS

Functional Programming à la 1940s

- Minimalist: who needs booleans?
- A boolean just *makes a choice!*

```
true   x y = x  
false x y = y
```

- We can *define* if-then-else!

```
ifte bool t e =  
  bool t e
```

Who needs integers?

- A (positive) integer just *counts loop iterations!*

two f x = f (f x)

one f x = f x

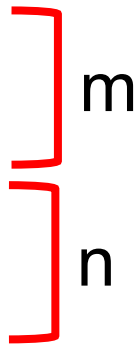
zero f x = x

- To recover a "normal" integer...

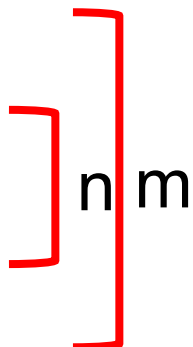
```
*Church> two (+1) 0  
2
```

Look, Ma, we can add!

- Addition by *sequencing* loops

add m n f x = m f (n f x) 

- Multiplication by *nesting* loops!

mul m n f x = m (n f) x 

```
*Church> add one (mul two two) (+1) 0  
5
```

Factorial à la 1940

```
fact n =  
  ifte (iszero n)  
    one  
    (mul n (fact (decr n)))
```

```
*Church> fact (add one (mul two two)) (+1) 0  
120
```

A couple more auxiliaries

- Testing for zero

```
iszero n =  
  n (\_ -> false) true
```

- Decrementing...

```
decr n =  
  n (\m f x-> f (m incr zero))  
    zero  
    (\x->x)  
    zero
```

Booleans, integers, (and other data structures) *can be entirely replaced by functions!*

"Church encodings"

Early versions of the Glasgow Haskell compiler actually implemented data-structures this way!



Alonzo Church

Before you try this at home...

Church.hs:27:35:

Occurs check: cannot construct the infinite type:

$$t \sim t \rightarrow t \rightarrow t$$

Actual type:

Expected type:

[illegible][illegible]

But wait, there's more...

Relevant bindings include

[illegible][illegible]

```

-> (((((t -> t) -> t) -> t) -> t) -> t) -> t -> t -> t)
-> (((t -> t -> t) -> t) -> t) -> (t -> t -> t) -> t -> t -> t)
-> ((t -> t -> t) -> t -> t)
-> t
-> t
-> t)
-> (((((t -> t -> t) -> t -> t) -> (t -> t -> t) -> t -> t -> t)
-> ((t -> t -> t) -> t -> t) -> (t -> t -> t) -> t -> t -> t)
-> (((t -> t -> t) -> t -> t) -> t -> t -> t)
-> (t -> t -> t)
-> t
-> t
-> t)
-> ((((((t -> t -> t) -> t -> t) -> (t -> t -> t) -> t -> t -> t)
-> ((t -> t -> t) -> t -> t) -> (t -> t -> t) -> t -> t -> t)
-> ((t -> t -> t) -> t -> t) -> (t -> t -> t) -> t -> t -> t)
-> (t -> t -> t)
-> t
-> t
-> t)
-> ((((((t -> t -> t) -> t -> t) -> (t -> t -> t) -> t -> t -> t)
-> (((((t -> t -> t) -> t -> t) -> (t -> t -> t) -> t -> t -> t)
-> ((t -> t -> t) -> t -> t) -> (t -> t -> t) -> t -> t -> t)
-> ((t -> t -> t) -> t -> t) -> (t -> t -> t) -> t -> t -> t)
-> (t -> t -> t)
-> t
-> t
-> t)
-> ((((((t -> t -> t) -> t -> t) -> (t -> t -> t) -> t -> t -> t)
-> (((t -> t -> t) -> t -> t) -> (t -> t -> t) -> t -> t -> t)
-> (((t -> t -> t) -> t -> t) -> (t -> t -> t) -> t -> t -> t)
-> ((t -> t -> t) -> t -> t) -> (t -> t -> t) -> t -> t -> t)
-> (t -> t -> t)
-> t
-> t
-> t)

```

(bound at Church.hs:27:1)

In the first argument of 'mul', namely 'n'

In the third argument of 'ifte', namely '(mul n (fact (decr n)))'

The type-checker needs a *little bit* of help

```
fact ::  
  (forall a. (a->a) ->a->a) ->  
  (a->a) -> a -> a
```

Factorial à la 1960



```
(LABEL FACT (LAMBDA (N)
  (COND ((ZEROP N) 1)
        (T (TIMES N (FACT (SUB1 N)))))))
```

Higher-order functions!

```
(MAPLIST FACT (QUOTE (1 2 3 4 5)))

(1 2 6 24 120)
```

The Next 700 Programming Languages

P. J. Landin

Univac Division of Sperry Rand Corp., New York, New York

“... today ... 1,700 special programming languages used to ‘communicate’ in over 700 application areas.”—*Computer Software Issues*, an American Mathematical Association Prospectus, July 1965.



Factorial in ISWIM

fac(5)

where rec fac(n) =

(n=1) → 1;

n*fac(n-1)

Laws

$(\text{MAPLIST } F \ (\text{REVERSE } L)) \equiv (\text{REVERSE } (\text{MAPLIST } F \ L))$

What's the point of two different ways to do the same thing?

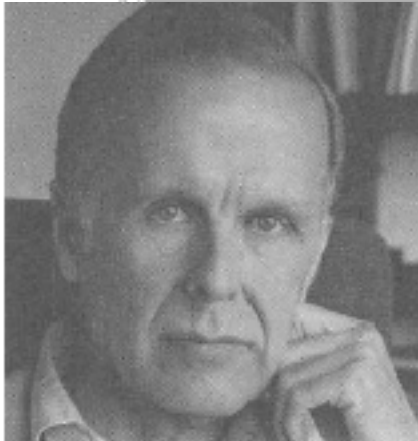
Wouldn't *two* facilities be better than one?

Expressive power should be by design, rather than by accident!



Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs

John Backus
IBM Research Laboratory, San Jose



Turing award 1977

[Paper 1978](#)

**Conventional programming
languages are growing ever
more enormous,
but not stronger.**

**Inherent defects at the most
basic level cause them to be
both **fat** and **weak**:**

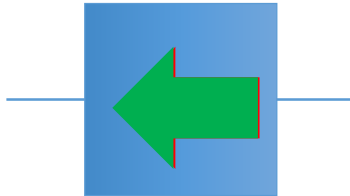
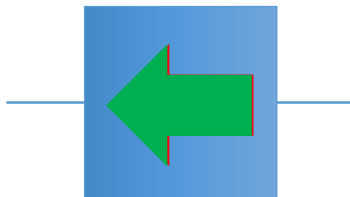
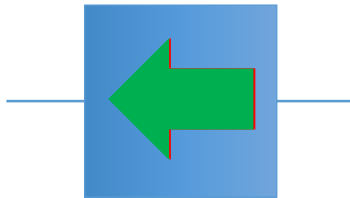
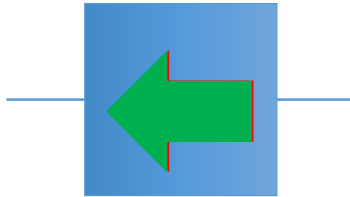
Word-at-a-time



their inability to effectively use
powerful combining forms
for building new programs from
existing ones



apply to all



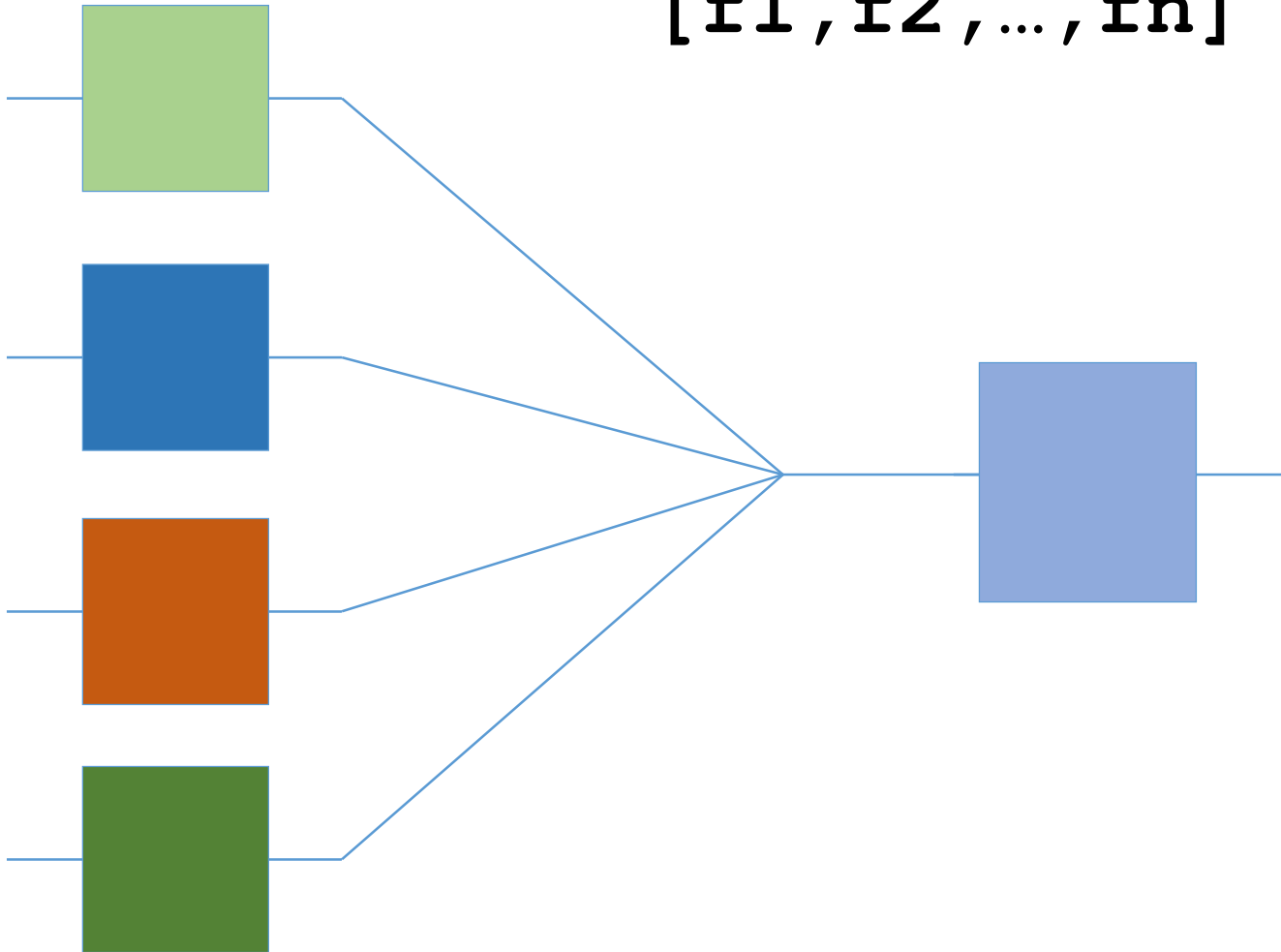
αf

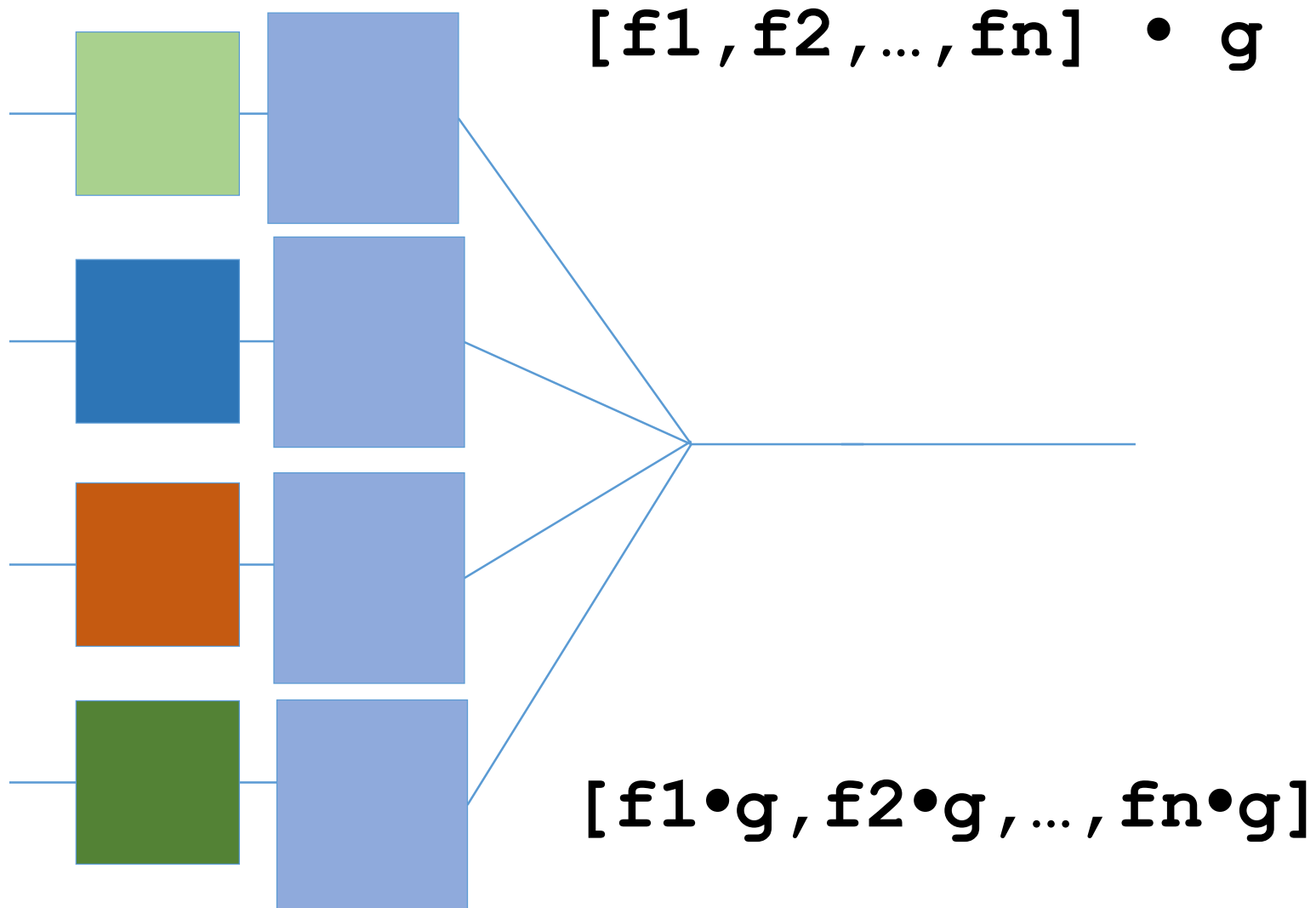
construction



**their lack of useful
mathematical properties for
reasoning about programs**

$[f_1, f_2, \dots, f_n] \bullet g$



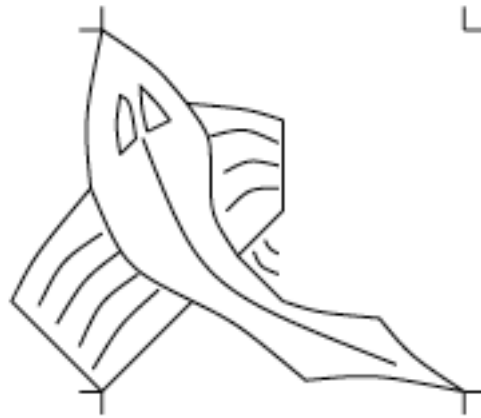



```
c := 0;  
for i := 1 step 1 until n do  
    c := c + a[i] × b[i]
```

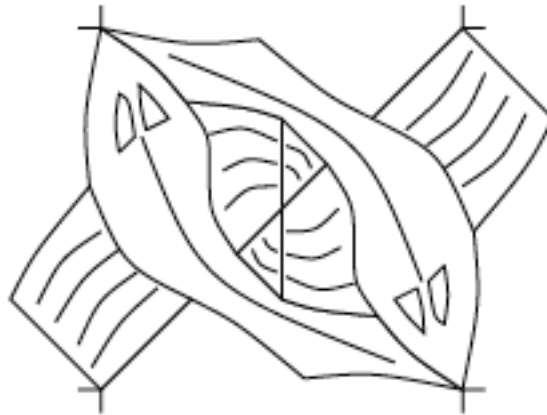
Def IP = (/ +) • (α ×) • Trans

Peter Henderson, Functional Geometry, 1982

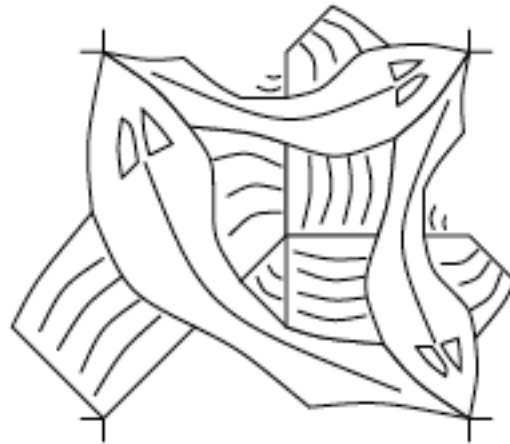




fish



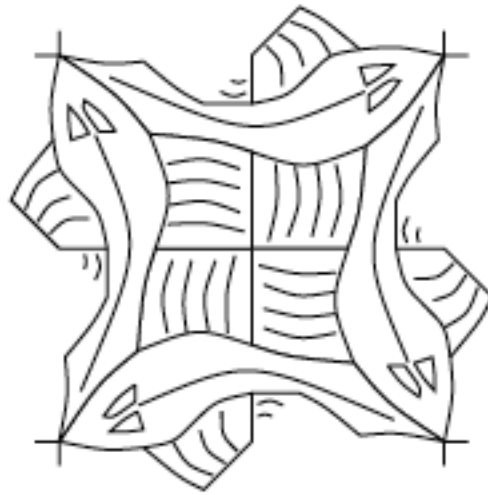
`over (fish, rot (rot (fish)))`



```
t = over (fish, over (fish2, fish3))
```

```
fish2 = flip (rot45 fish)
```

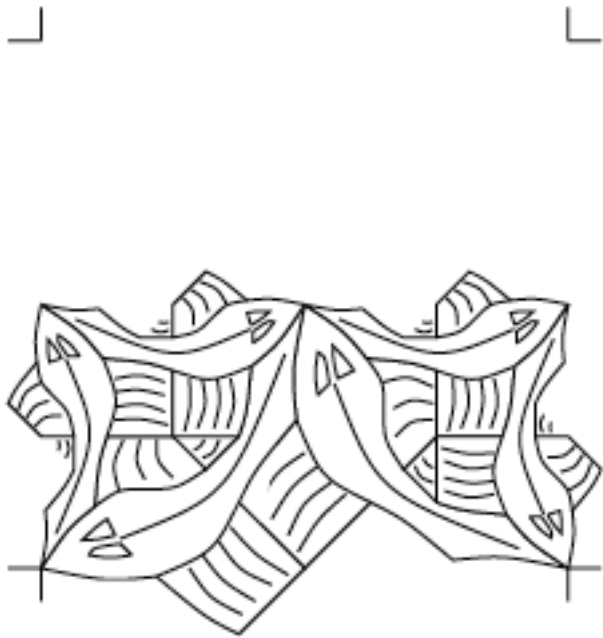
```
fish3 = rot (rot (rot (fish2)))
```



```
u = over (over (fish2, rot (fish2)),  
          over (rot (rot (fish2)),  
                rot (rot (rot (fish2))))))
```

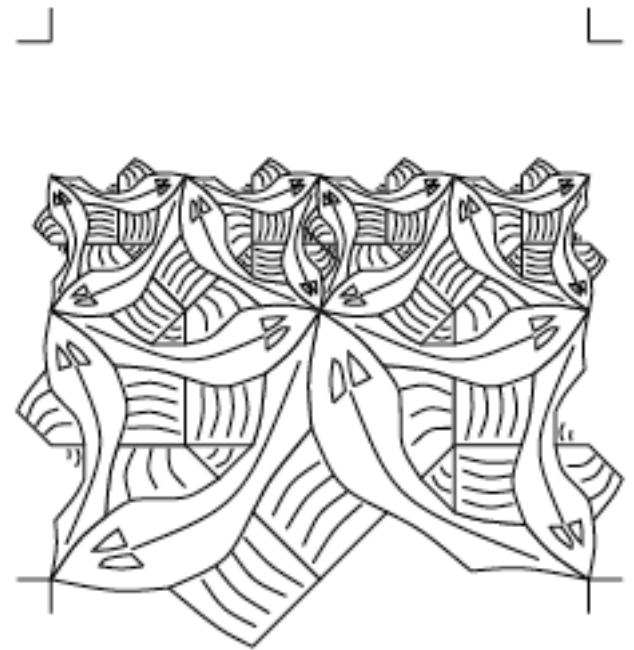
P	Q
R	S

quartet



```
quartet(nil, nil,
        rot(t), t)
```

side1

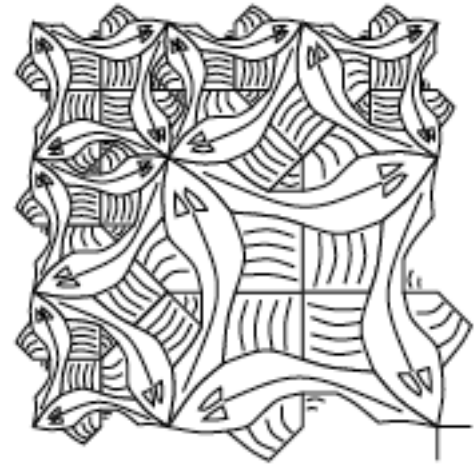


```
quartet(side1, side1,
        rot(t), t )
```



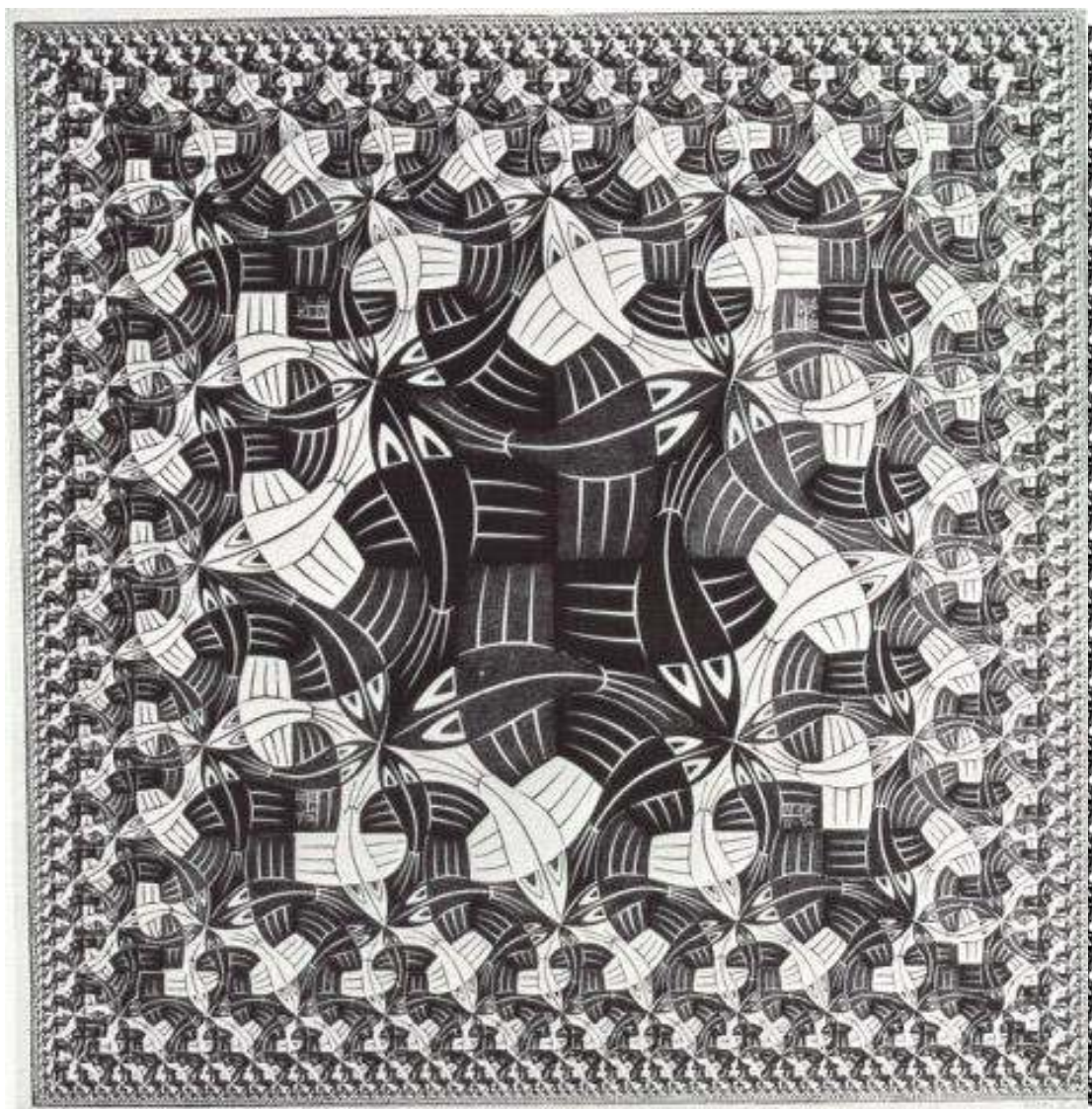
quartet (nil,nil,nil,u)

corner1



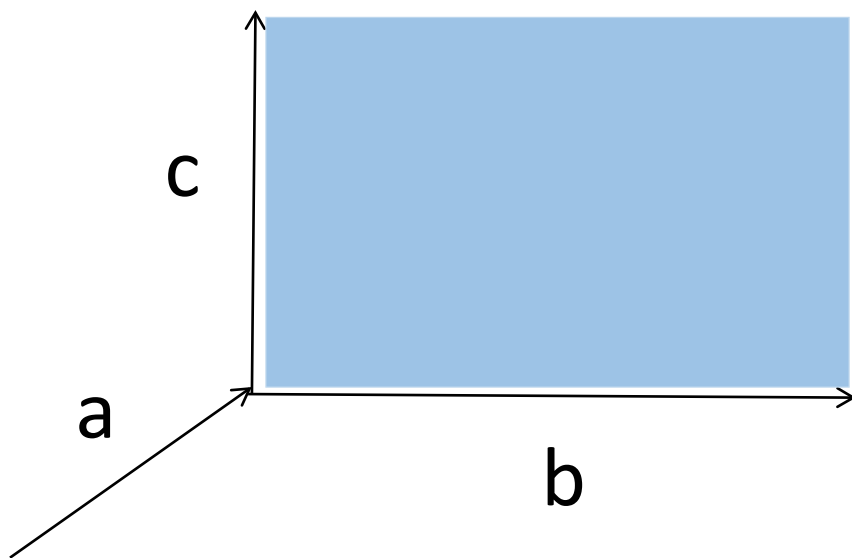
quartet (corner1,
side1,
rot(side1),
u)

```
squarelimit = nonet(  
    corner,      side,      rot(rot(rot(corner))),  
    rot(side),  u,          rot(rot(rot(side))),  
    rot(corner), rot(rot(side)), rot(rot(corner)))
```

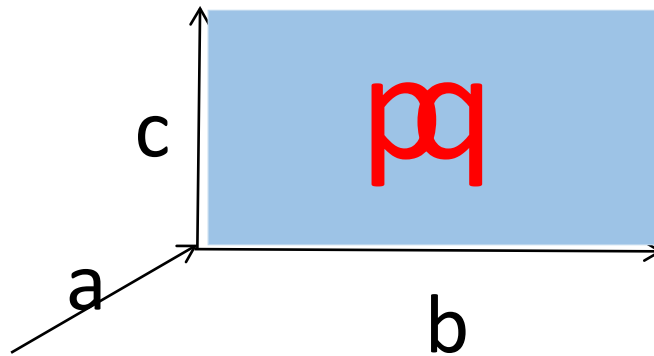


picture = function

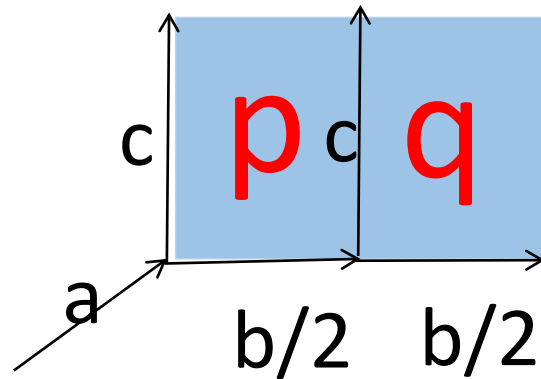
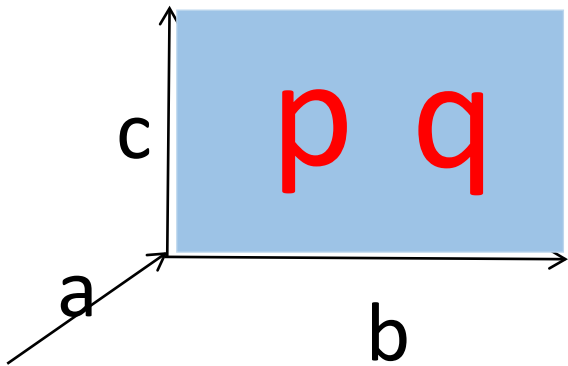
picture = function



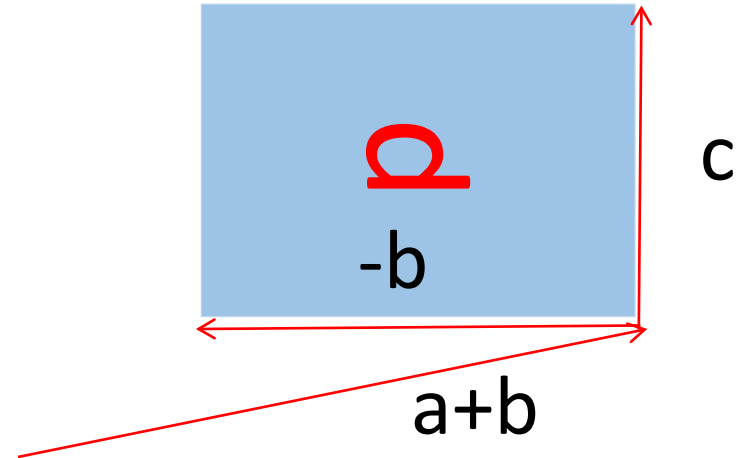
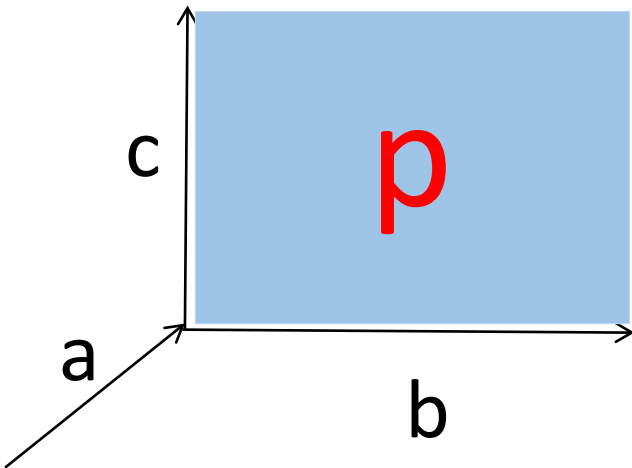
$$\text{over } (p, q) \quad (a, b, c) = \\ p(a, b, c) \cup q(a, b, c)$$



$$\text{beside } (p, q) \ (a, b, c) = \\ p(a, b/2, c) \cup q(a+b/2, b/2, c)$$



$$\text{rot}(p) \ (a, b, c) = p(a+b, c, -b)$$



Laws

$$\begin{aligned} &\text{rot}(\text{above}(p, q)) \\ &= \\ &\text{beside}(\text{rot}(p), \text{rot}(q)) \end{aligned}$$



It seems there is a positive correlation between the simplicity of the rules and the quality of the algebra as a description tool.

Whole values

Combining forms

Algebra as litmus test

Functions as representations

Haskell vs. Ada vs. C++ vs. Awk vs. ... An Experiment in Software Prototyping Productivity*

Paul Hudak
Mark P. Jones

Yale University
Department of Computer Science
New Haven, CT 06518
`{hudak-paul, jones-mark}@cs.yale.edu`



July 4, 1994



Time 40.0:

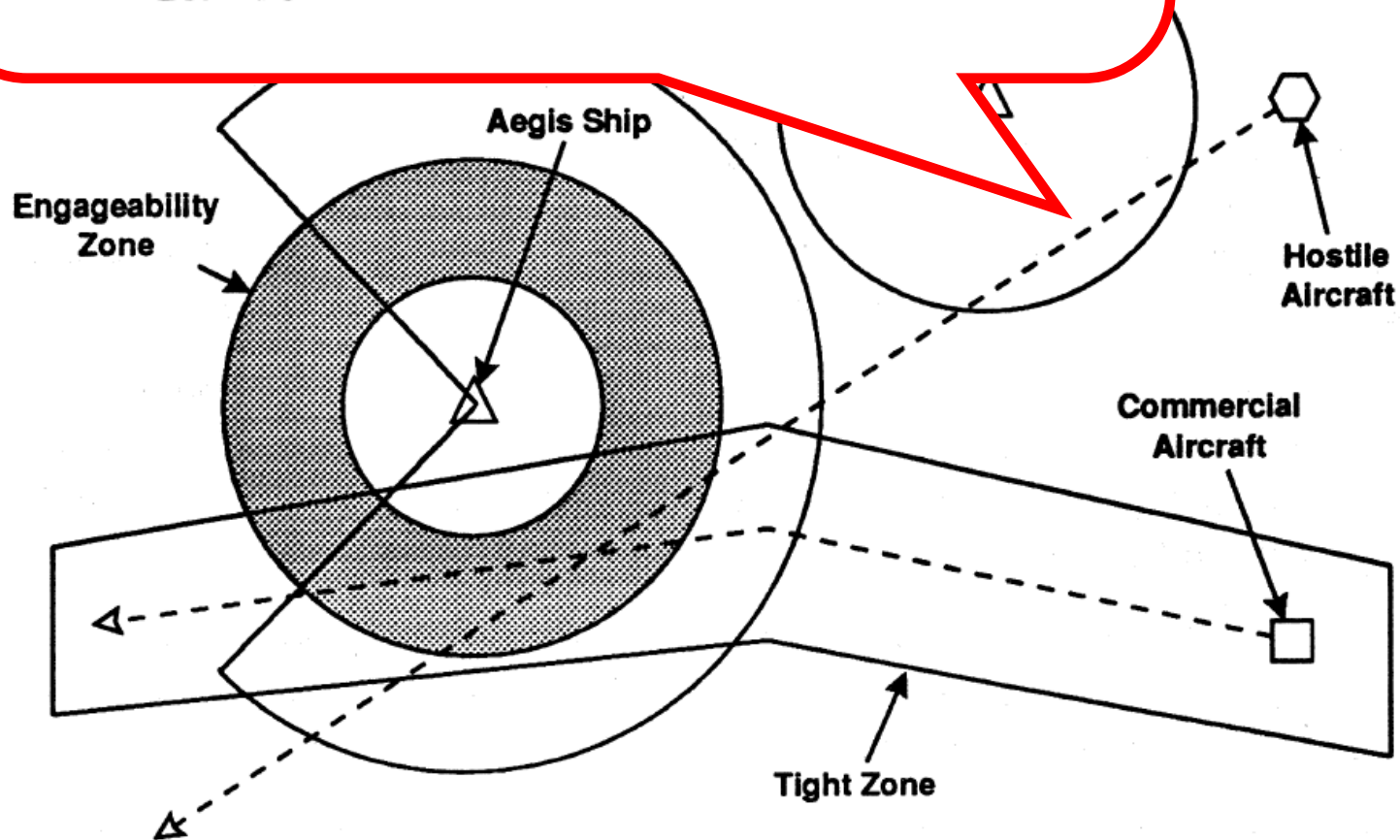
commercial aircraft: (100.0,43.0)

-- In engageability zone

-- In tight zone

hostile craft: (210.0,136.0)

-- In carrier slave doctrine



Functions as Data

```
> type Region    =    Point -> Bool
```

Including 29 lines of
inferable type
signatures/synonyms

A student, given 8 days
to learn Haskell, w/o
knowledge of Yale group

Language	Lines of code	Lines of document	Development time (hours)
(1) Haskell	85	465	10
(2) Ada	767	7	23
(3) Ada9X	800		28
(4) C++	1105	130	-
(5) Awk/Nawk	250	150	-
(6) Rapide	157	0	54
(7) Griffin	251	0	34
(8) Proteus	293	79	26
(9) Relational Lisp	274	12	3
(10) Haskell	156	112	8

Figure 3: Summary of Prototype Software Development Metrics

Reaction...

”too cute for its own good”

...higher-order functions just a trick, probably not useful in other contexts

Lazy Evaluation (1976)



Henderson and Morris
A lazy evaluator



Friedman and Wise
*CONS should not evaluate
its arguments*

“The Whole Value” can be ∞ !

- The *infinite list* of natural numbers
`[0, 1, 2, 3 ...]`

Consumer decides
how many to
compute

- All the iterations of a function
`iterate f x = [x, f x, f (f x), ...]`

- A consumer for numerical methods

`limit eps xs =`

*<first element of **xs** within **eps** of its predecessor>*

Some numerical algorithms

- Newton-Raphson square root

```
sqrt a = limit eps (iterate next 1.0)  
  where next x = (x + a/x) / 2
```

The diagram for the Newton-Raphson square root algorithm shows two red ovals highlighting 'limit eps' in the function definition and 'where' in the definition of 'next'. A red arrow points from the 'where' oval to the 'limit eps' oval. A green oval highlights the entire function definition 'sqrt a = limit eps (iterate next 1.0)'. A green arrow points from this oval to the 'limit eps' oval. A blue sequence '[1, 1/2, 1/4, 1/8...]' is shown with a blue arrow pointing to the 'iterate' function in the function definition. A green arrow points from the 'iterate' function to the 'next' function definition.

- Derivatives

```
deriv f x =  
  limit eps (map slope (iterate (/2) 1.0))  
  where slope h = (f (x+h) - f x) / h
```

[1, 1/2, 1/4, 1/8...]

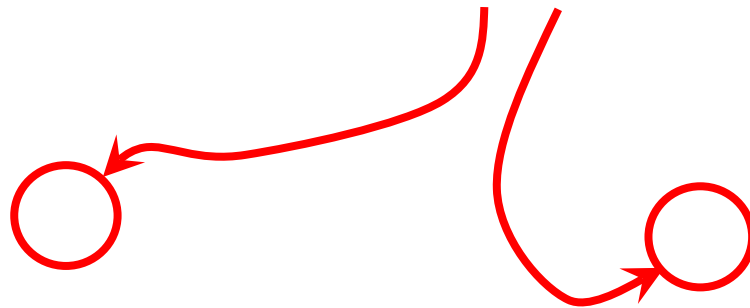
The diagram for the Derivatives algorithm shows two red ovals highlighting 'limit eps' in the function definition and 'where' in the definition of 'slope'. A red arrow points from the 'where' oval to the 'limit eps' oval. A green oval highlights the entire function definition 'deriv f x = limit eps (map slope (iterate (/2) 1.0))'. A green arrow points from this oval to the 'limit eps' oval. A blue sequence '[1, 1/2, 1/4, 1/8...]' is shown with a blue arrow pointing to the 'iterate' function in the function definition. A green arrow points from the 'iterate' function to the 'slope' function definition.

Same convergence check

Different approximation sequences

Speeding up convergence

The smaller h is, the better the approximation



Differentiation

Integration

The right
answer

$$A + B * h^n$$

An error term

Eliminating the error term

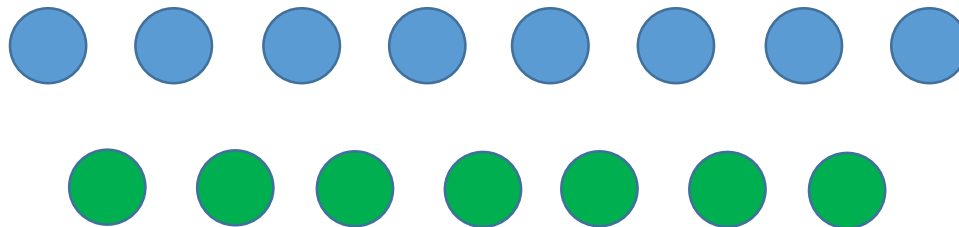
- Given:

$$A + B * h^n$$

$$A + B * (h/2)^n$$

Two successive approximations

- Solve for A and B!



improve n **xs** *converges faster than* **xs**

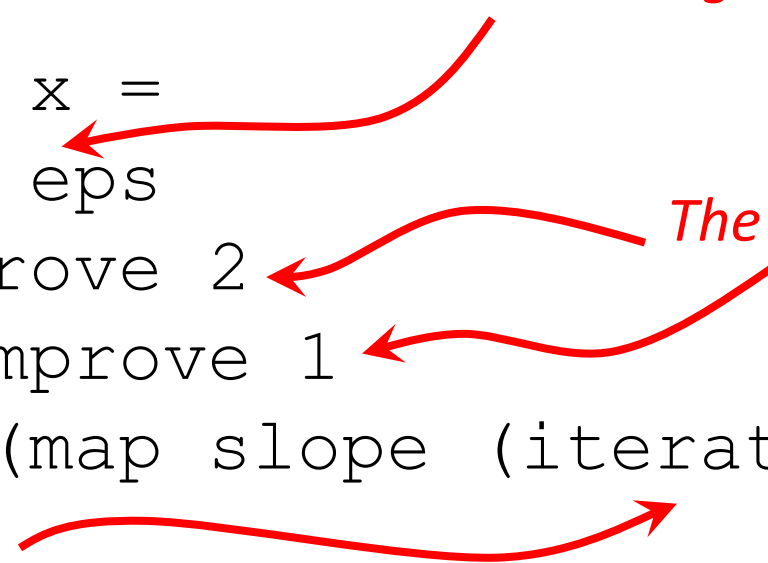
Really fast derivative

```
deriv f x =  
  limit eps  
    (improve 2  
      (improve 1  
        (map slope (iterate (/2) 1.0))))
```

The convergence check

The improvements

The approximations



Everything is programmed *separately* and easy to understand—thanks to “whole value programming”

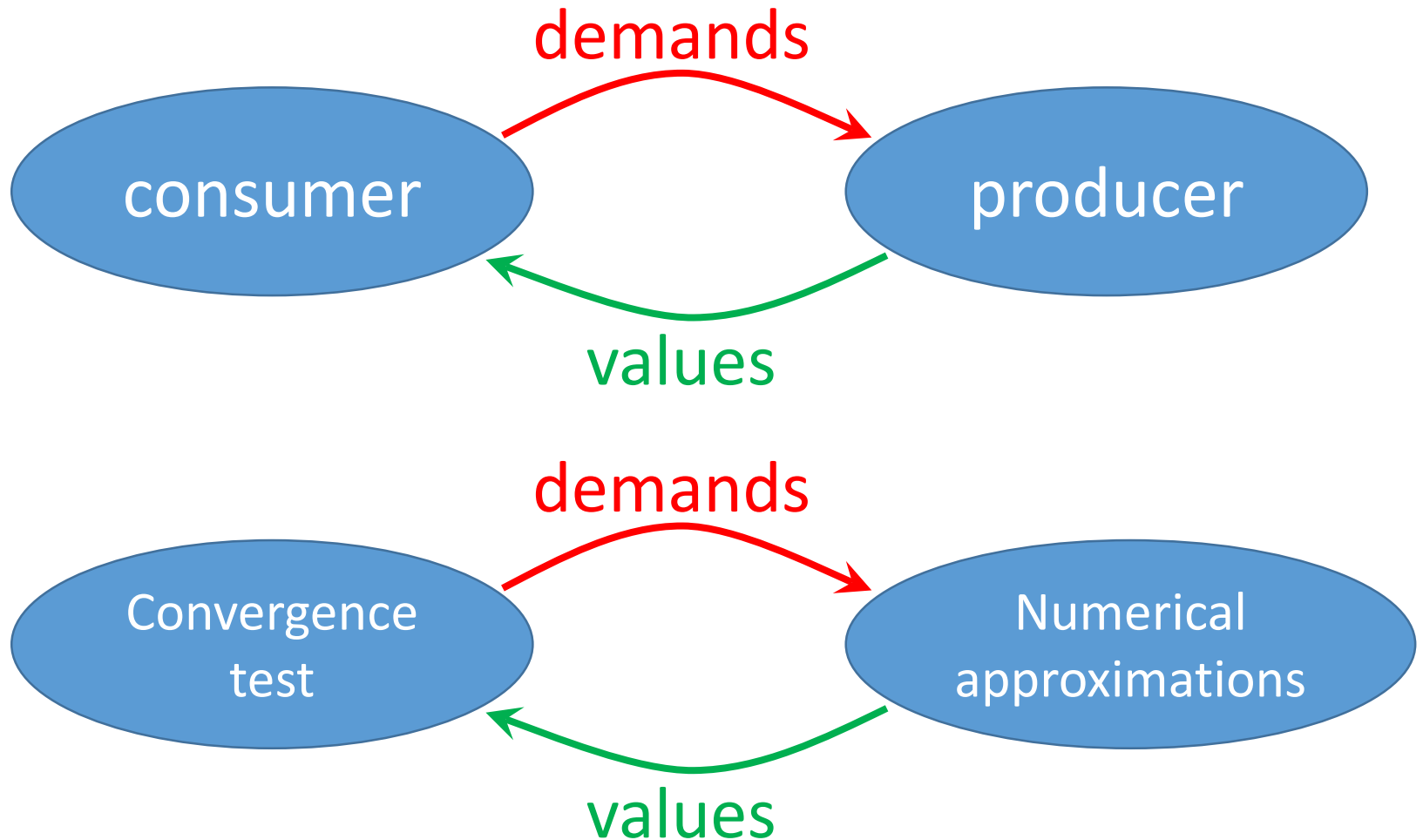
Why Functional Programming Matters

John Hughes
The University, Glasgow

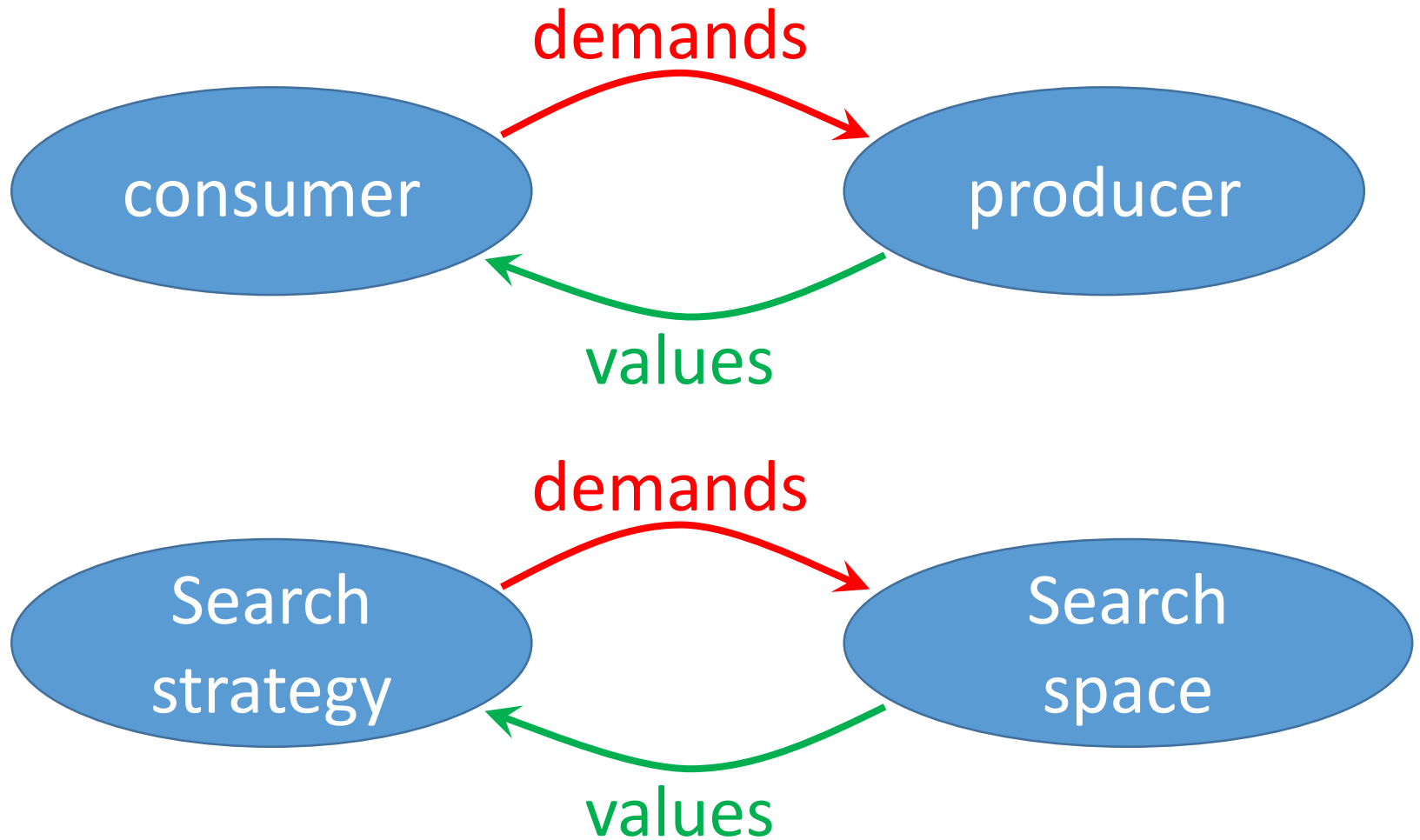


1990

Lazy producer-consumer

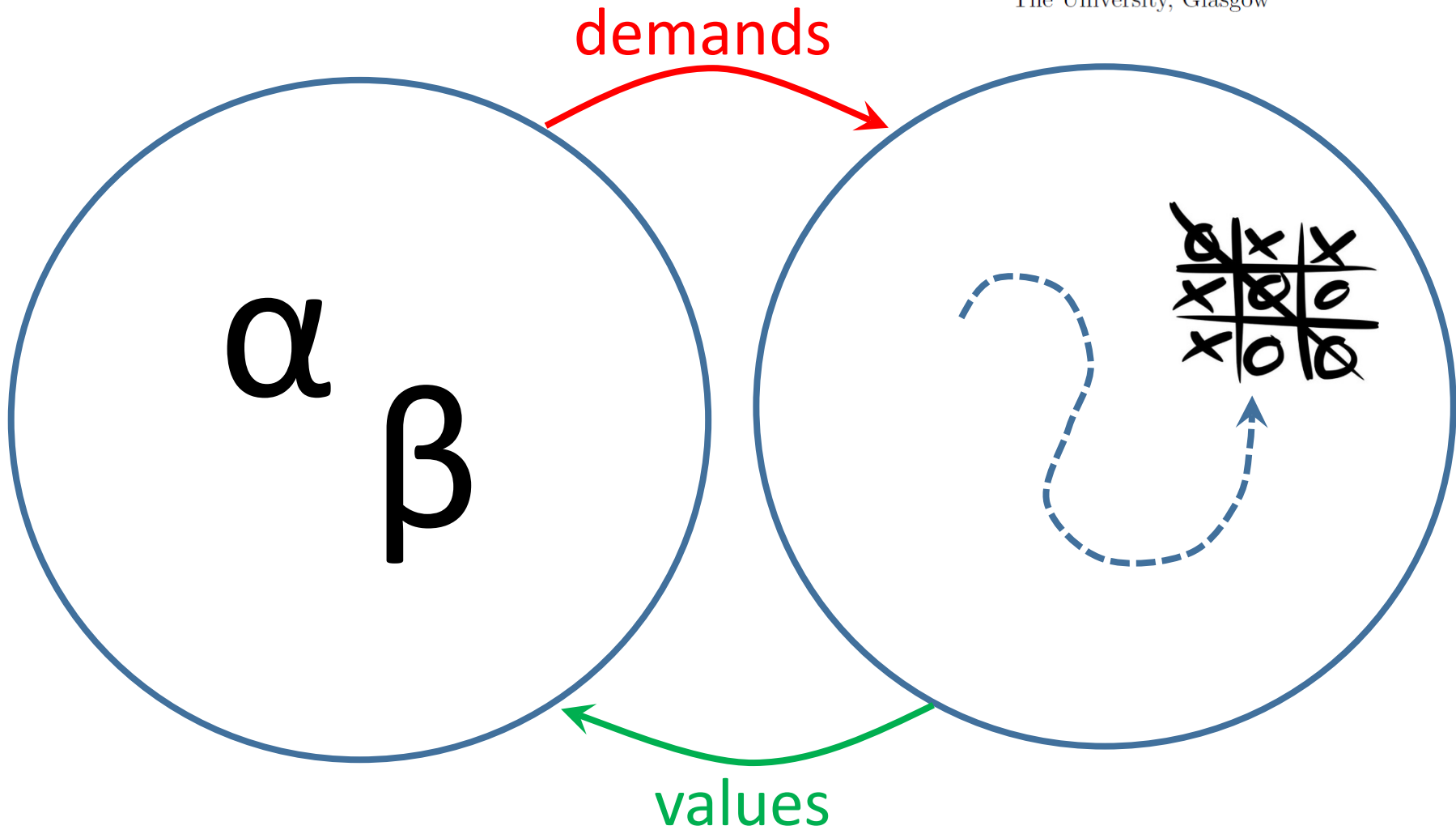


Lazy producer-consumer



Why Functional Programming Matters

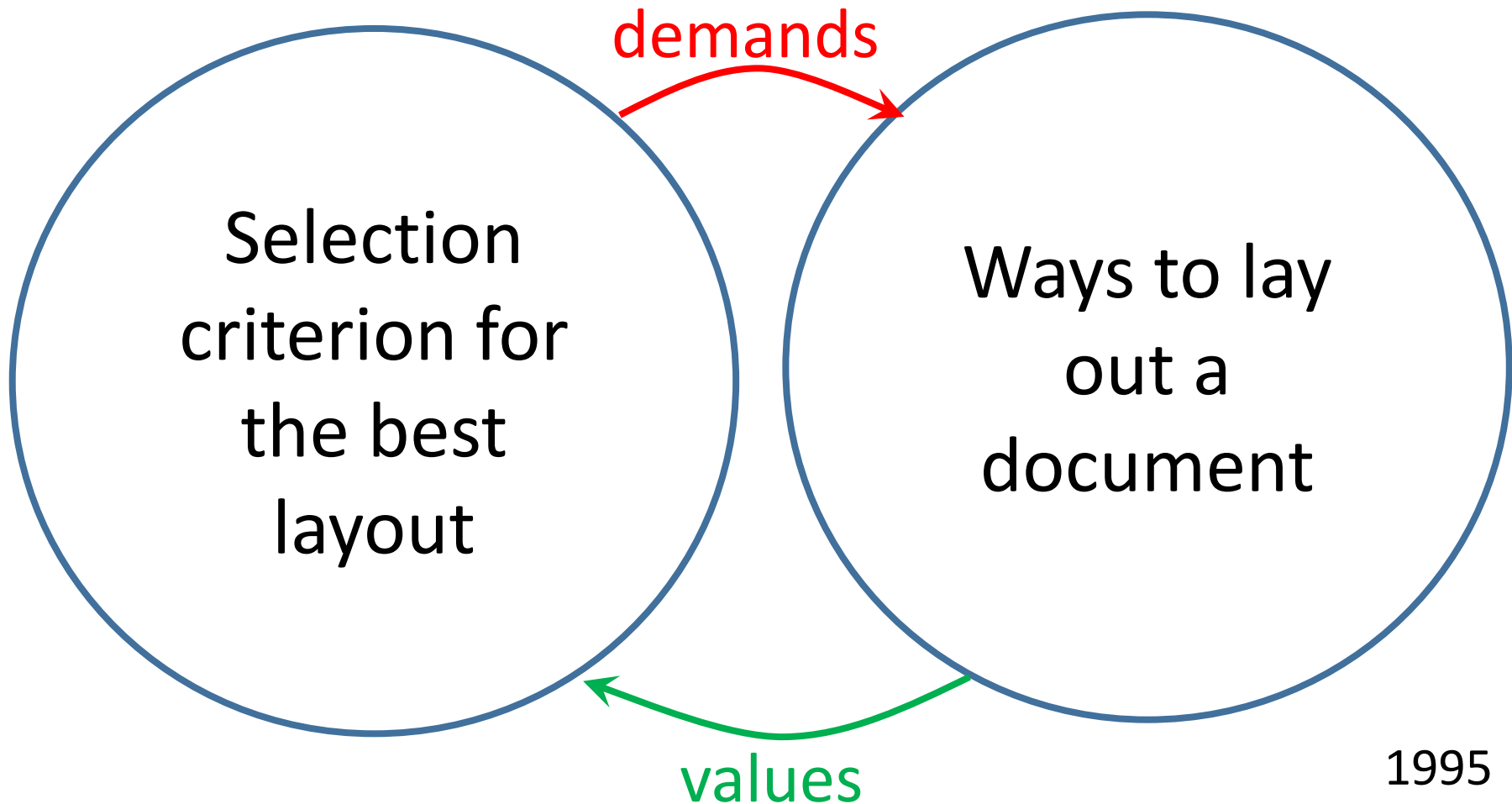
John Hughes
The University, Glasgow



The Design of a Pretty-printing Library

John Hughes

Chalmers Tekniska Högskola, Göteborg, Sweden.



Improved on by...





QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs



Koen Claessen
Chalmers University of Technology
koen@cs.chalmers.se

John Hughes
Chalmers University of Technology
rjmh@cs.chalmers.se

```
prop_reverse () ->  
  ?FORALL (Xs, list (int ()),  
           reverse (reverse (Xs)) == Xs) .
```

```
3> eqc:quickcheck (qc:prop_reverse ()) .
```

```
.....  
.....
```

```
OK, passed 100 tests
```

```
true
```



QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs



Koen Claessen
Chalmers University of Technology
koen@cs.chalmers.se

John Hughes
Chalmers University of Technology
rjmh@cs.chalmers.se

```
prop_wrong() ->  
  ?FORALL(Xs, list(int()),  
    reverse(Xs) == Xs) .
```

```
4> eqc:quickcheck(qc:prop_wrong()) .
```

```
Failed! After 1 tests.
```

```
[-36,-29,20,31,-47,-63,80,-7,93,-87,-29,33,64,58]
```

```
Shrinking xx.x.x..xx(4 times)
```

```
[0,1] ← minimal counterexample  
false
```

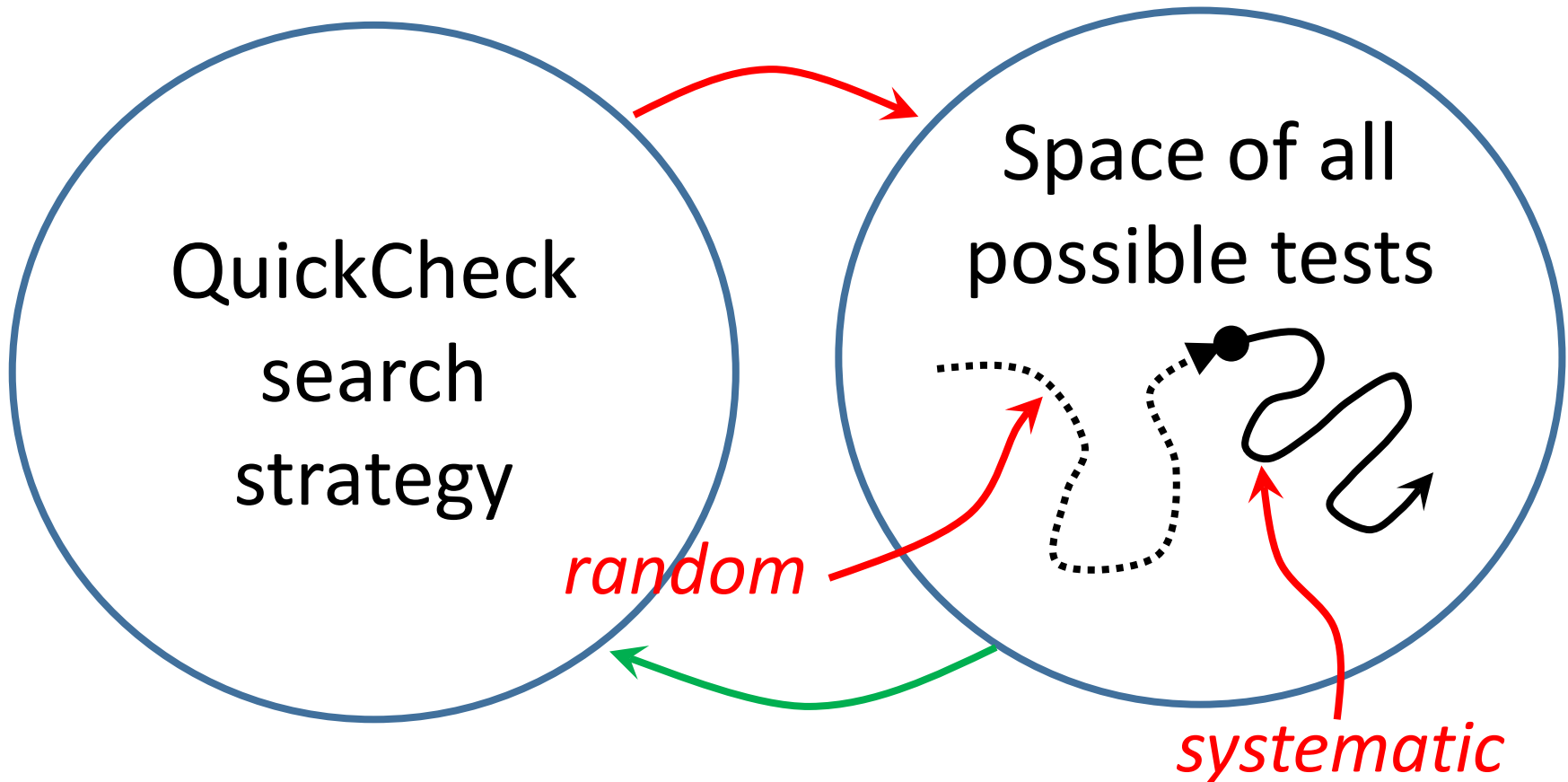


QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs



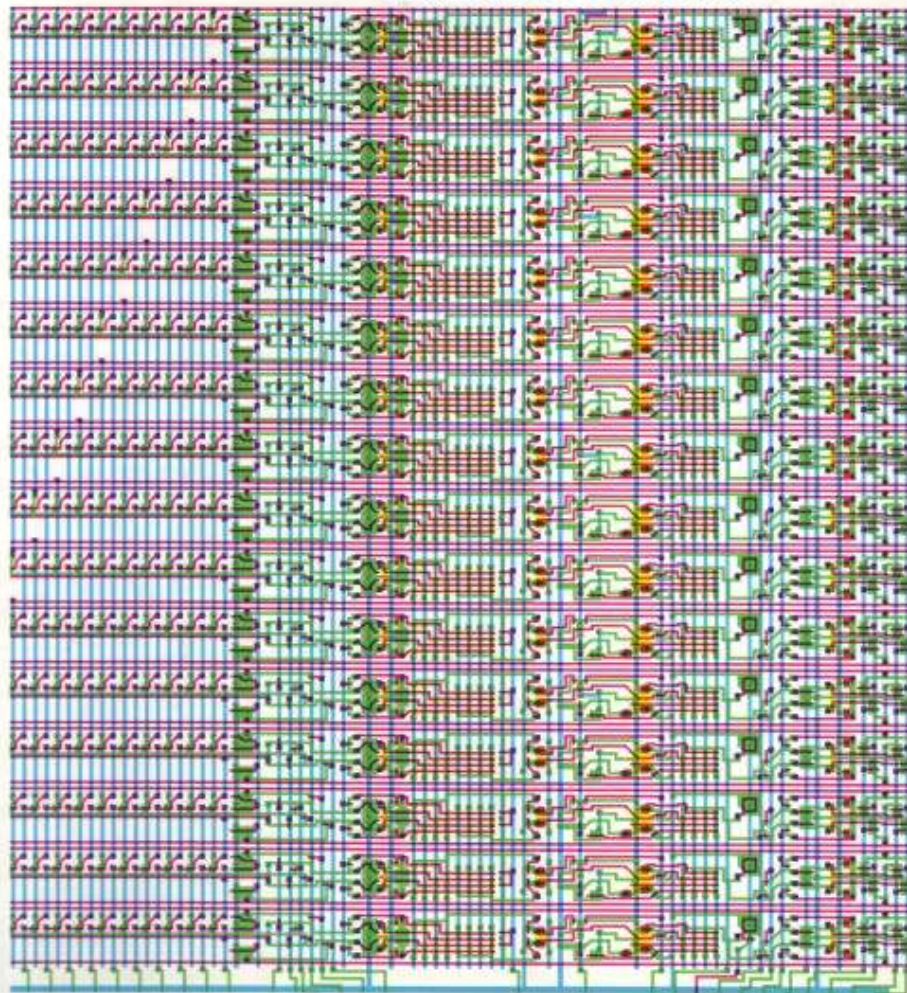
Koen Claessen
Chalmers University of Technology
koen@cs.chalmers.se

John Hughes
Chalmers University of Technology
rjmh@cs.chalmers.se



INTRODUCTION TO **VLSI** SYSTEMS

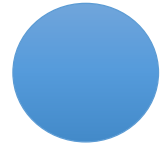
CARVER MEAD • LYNN CONWAY



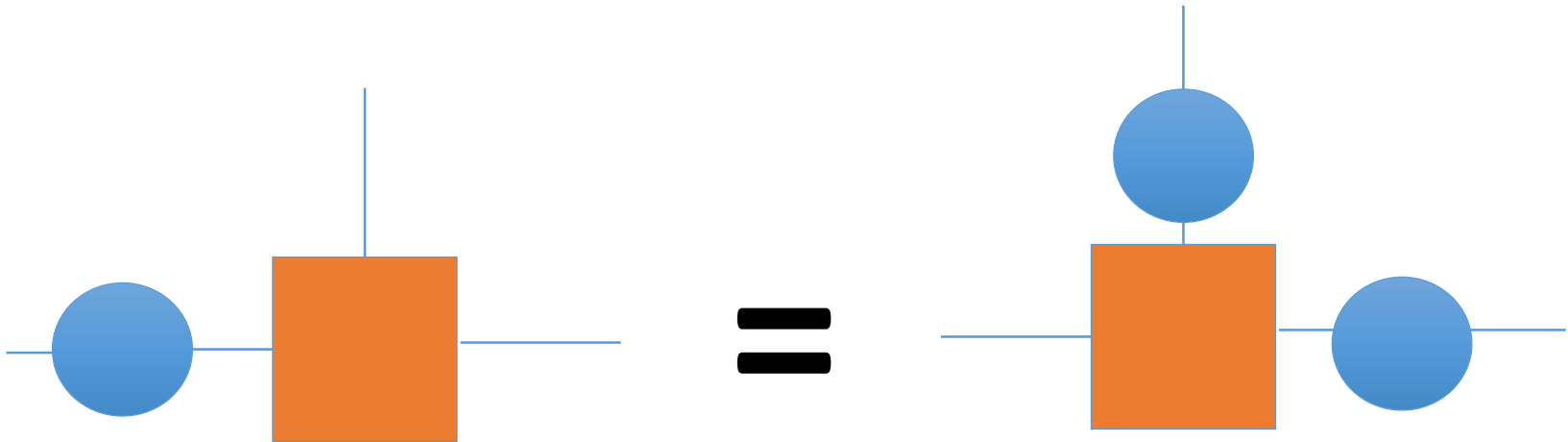
muFP—Circuits as values

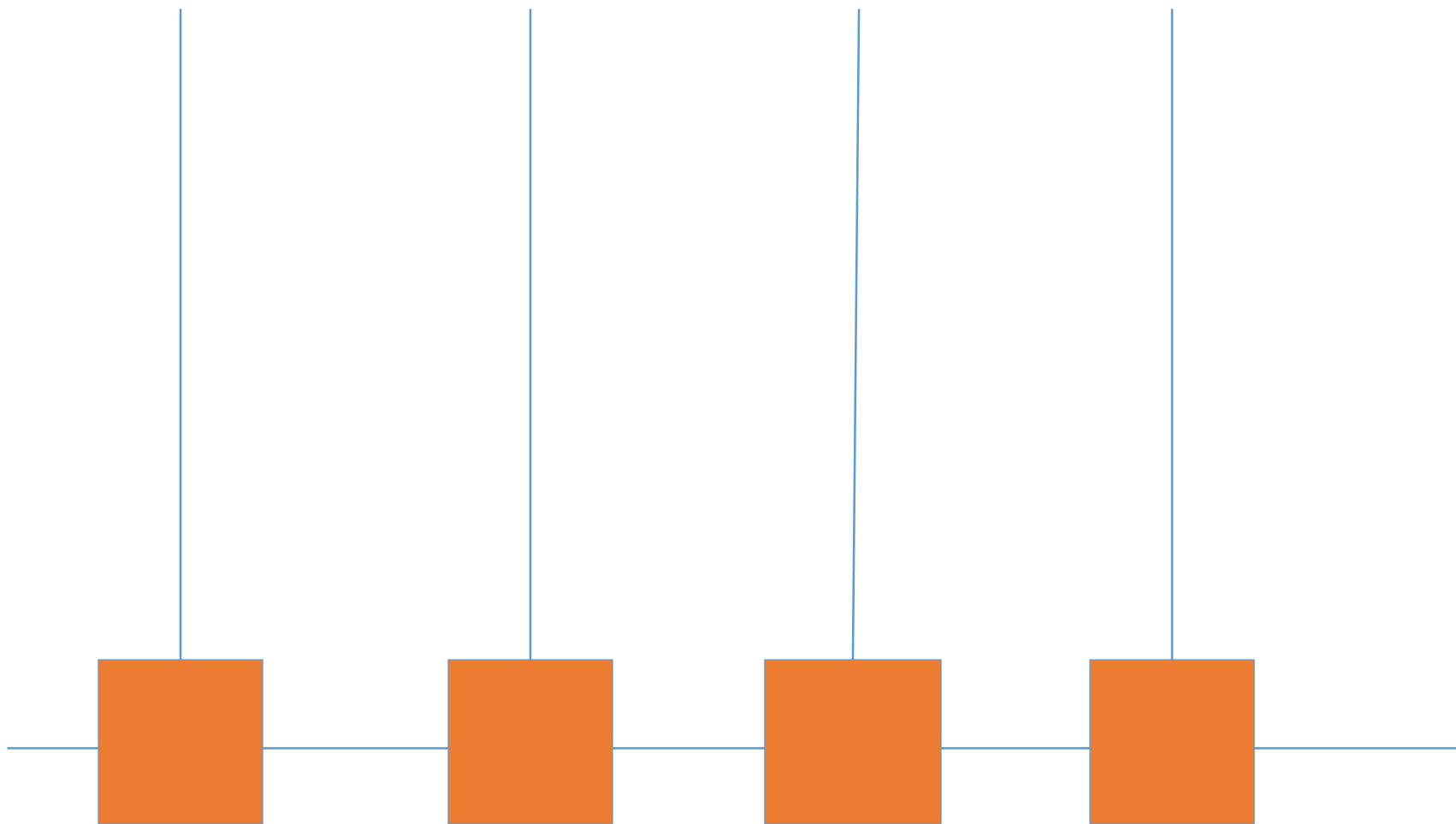


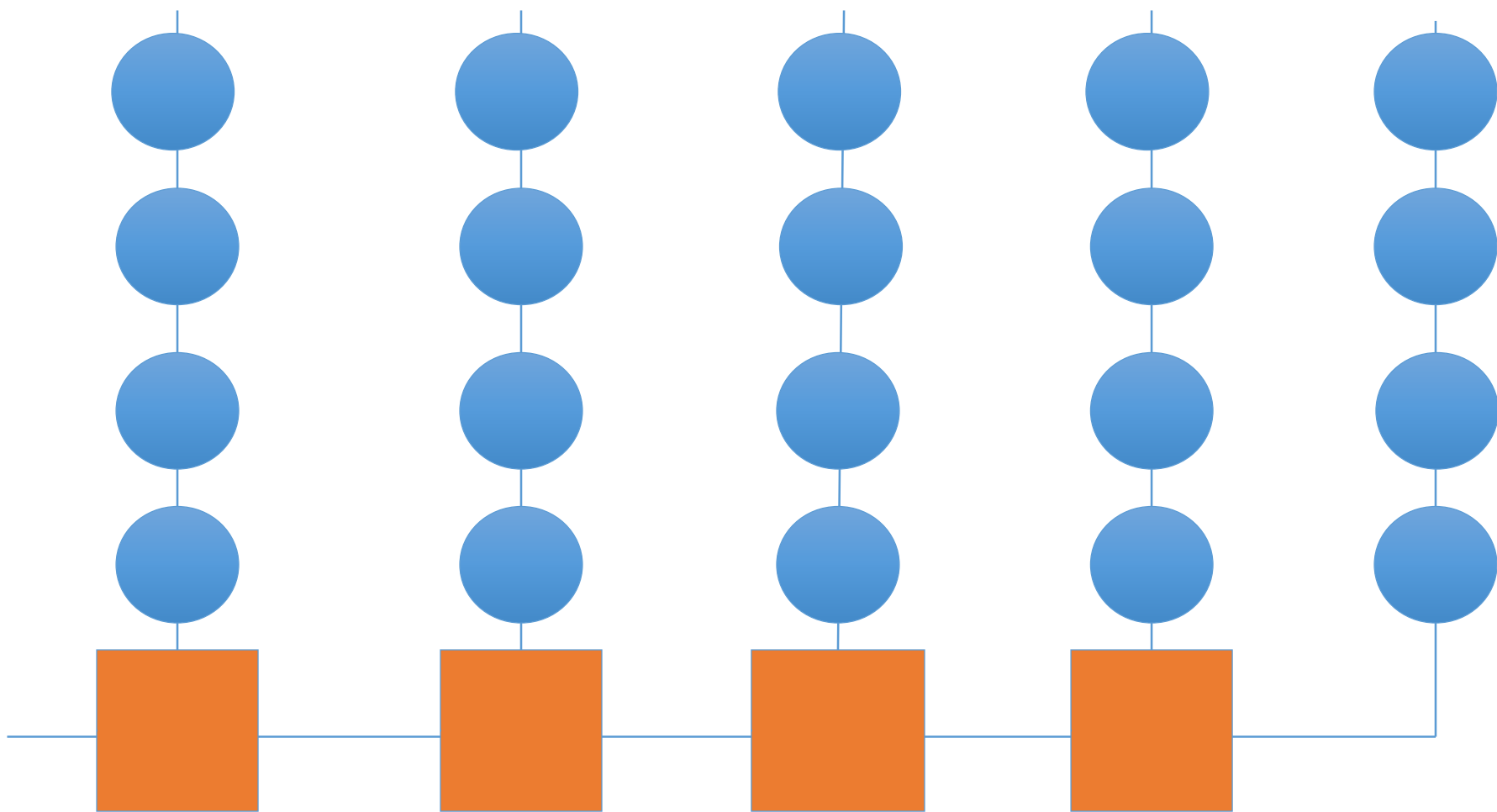
- Backus FP + unit delays
- Inherits many combining forms and laws
- Good for reasoning about alternative designs

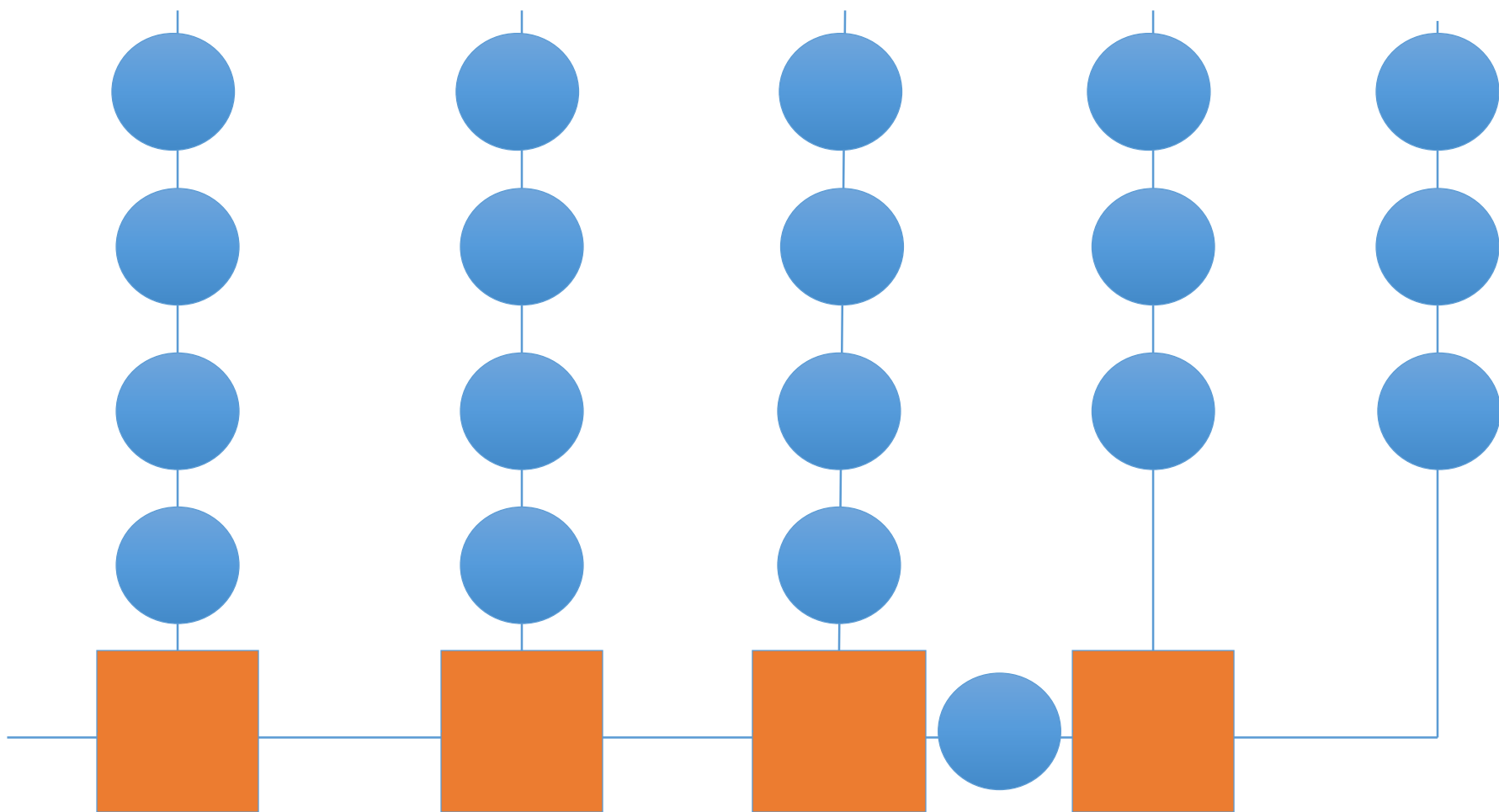


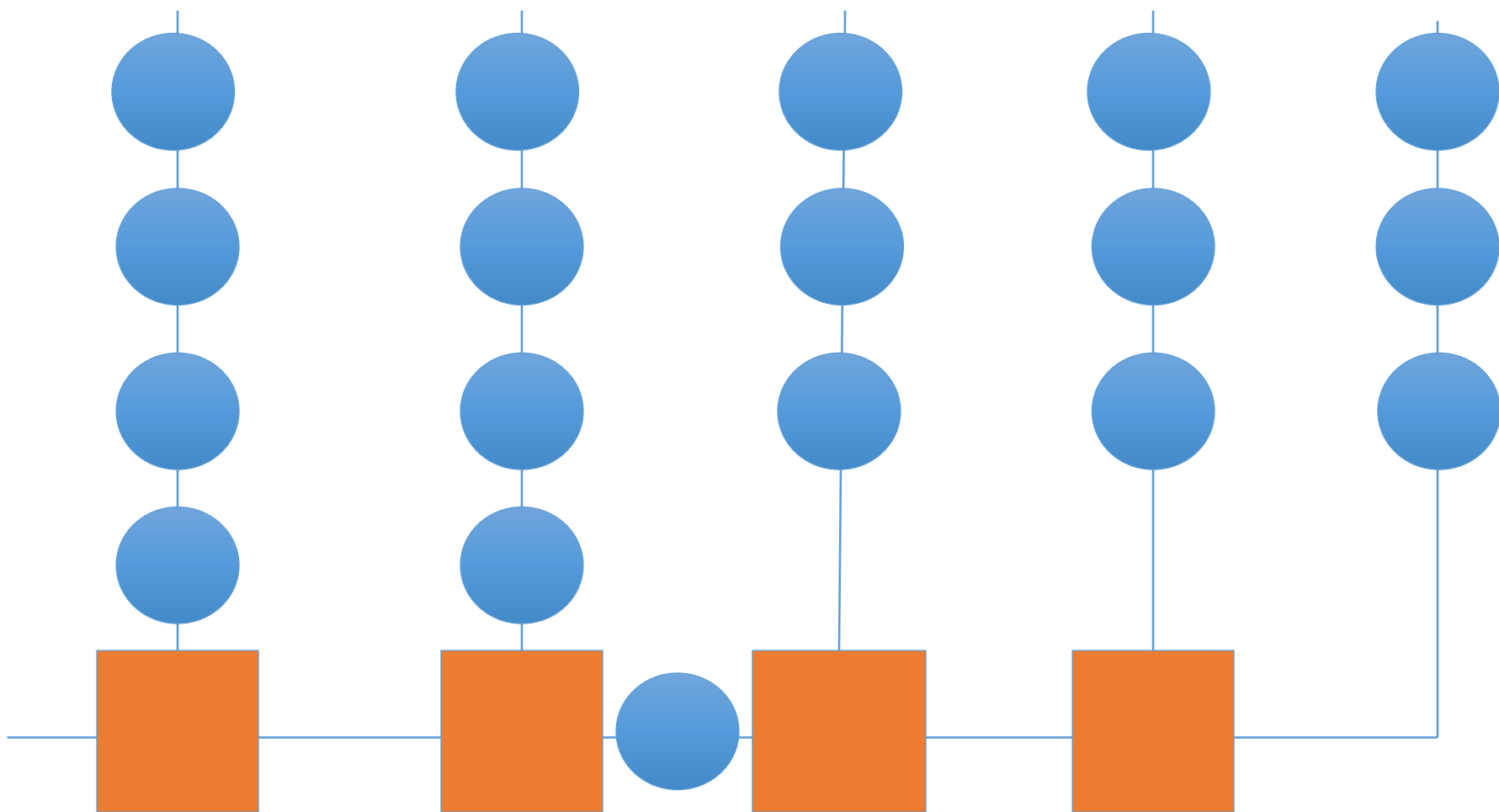
Example law: “retiming”

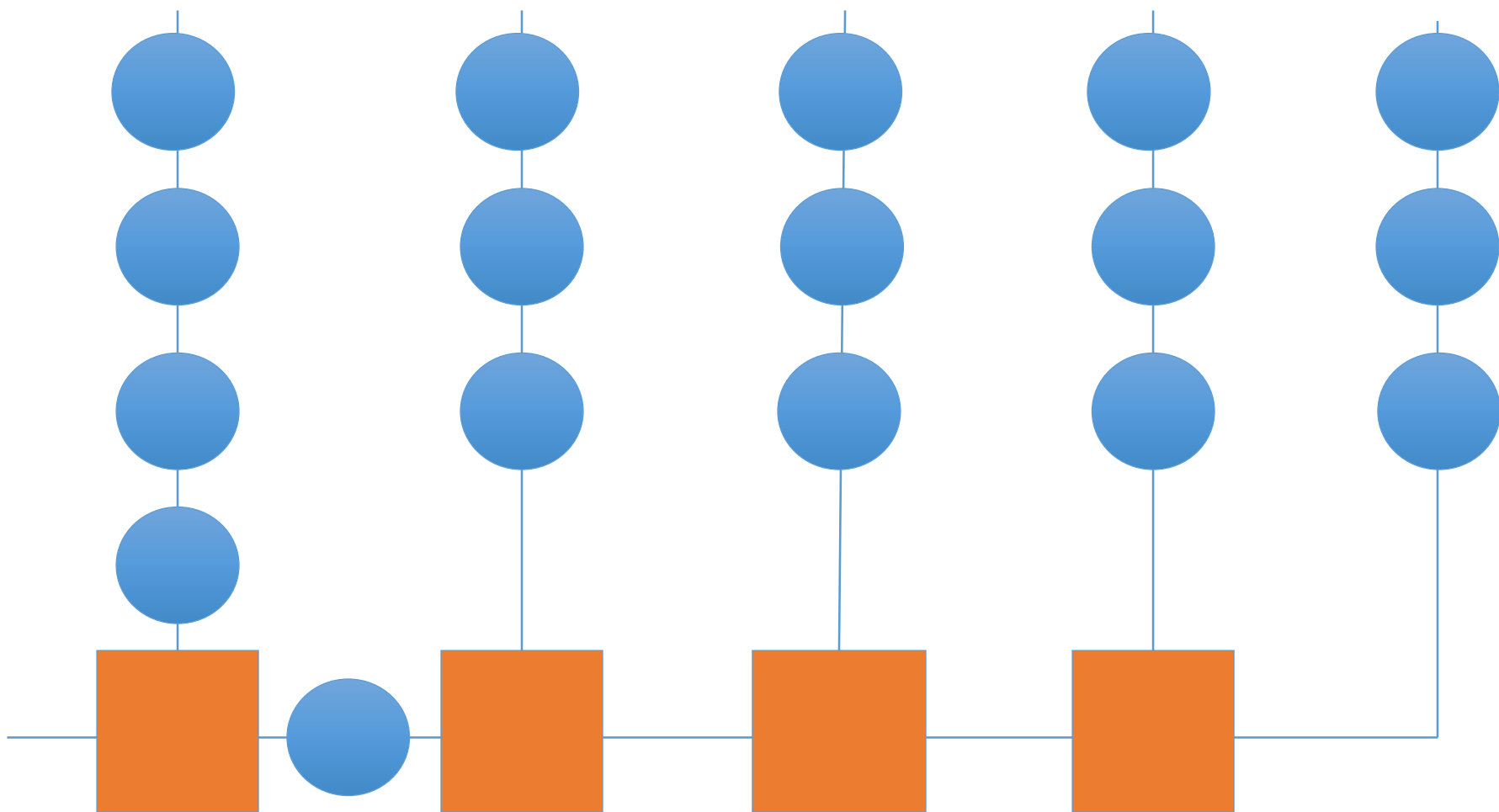


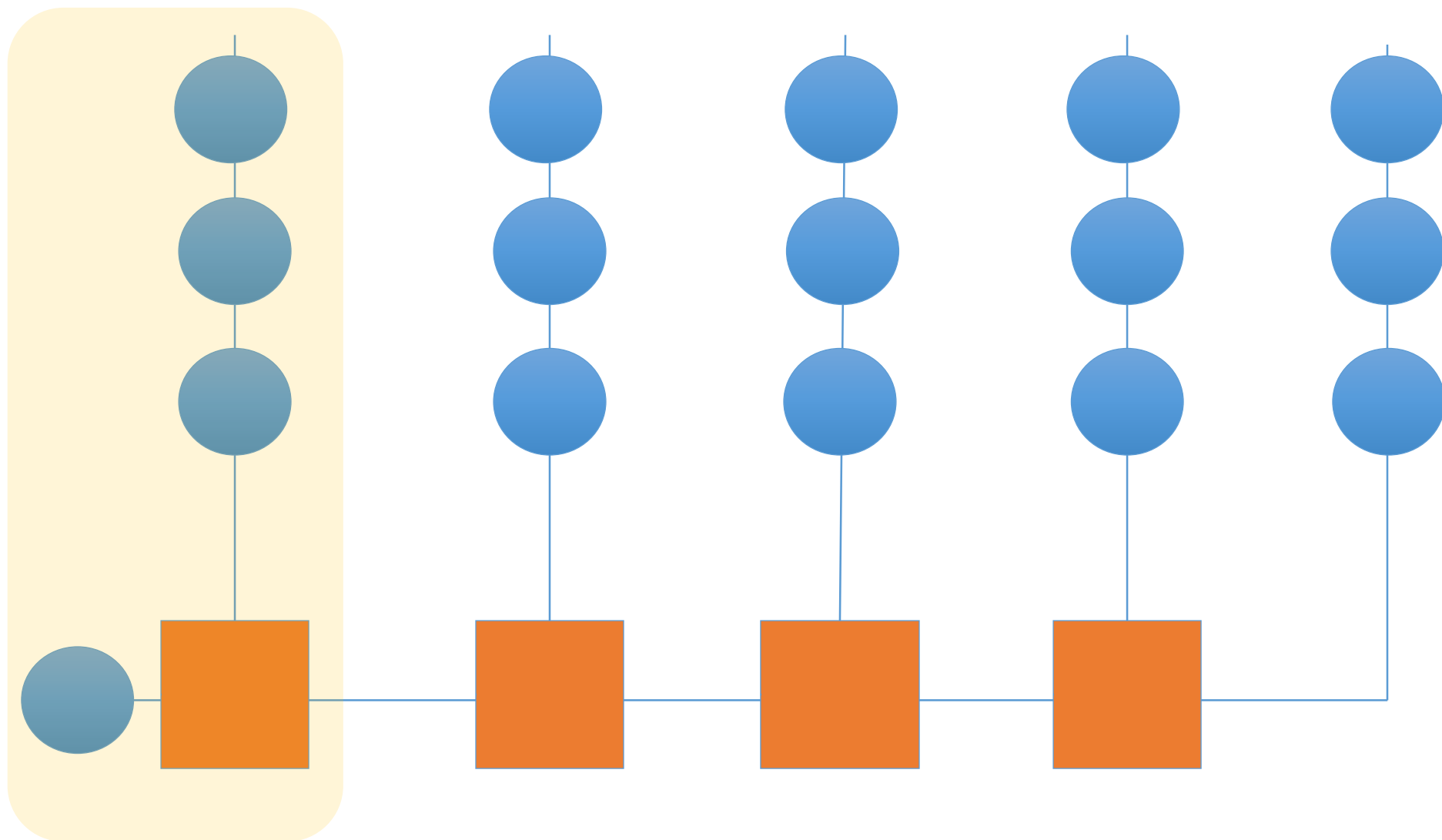


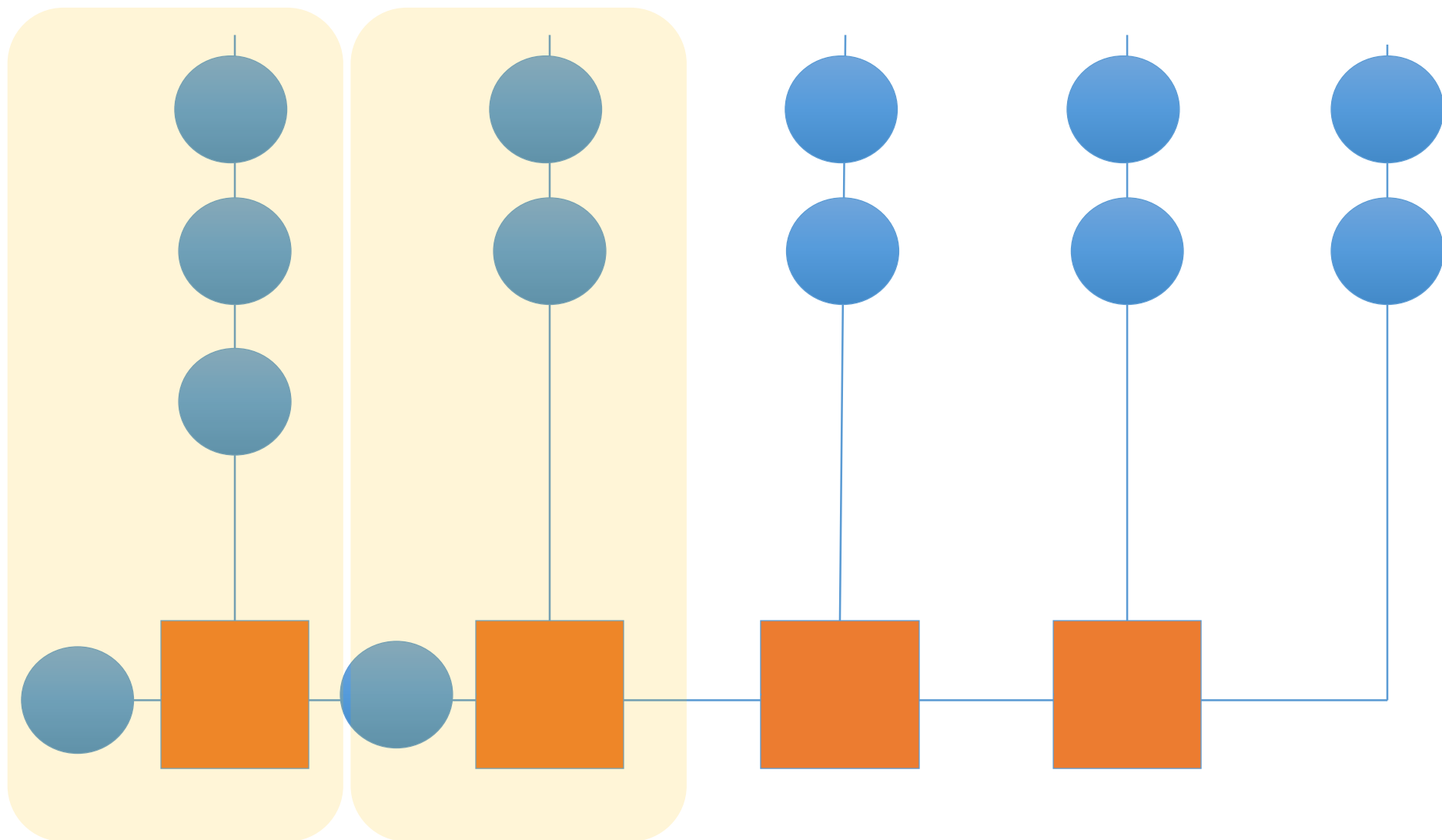


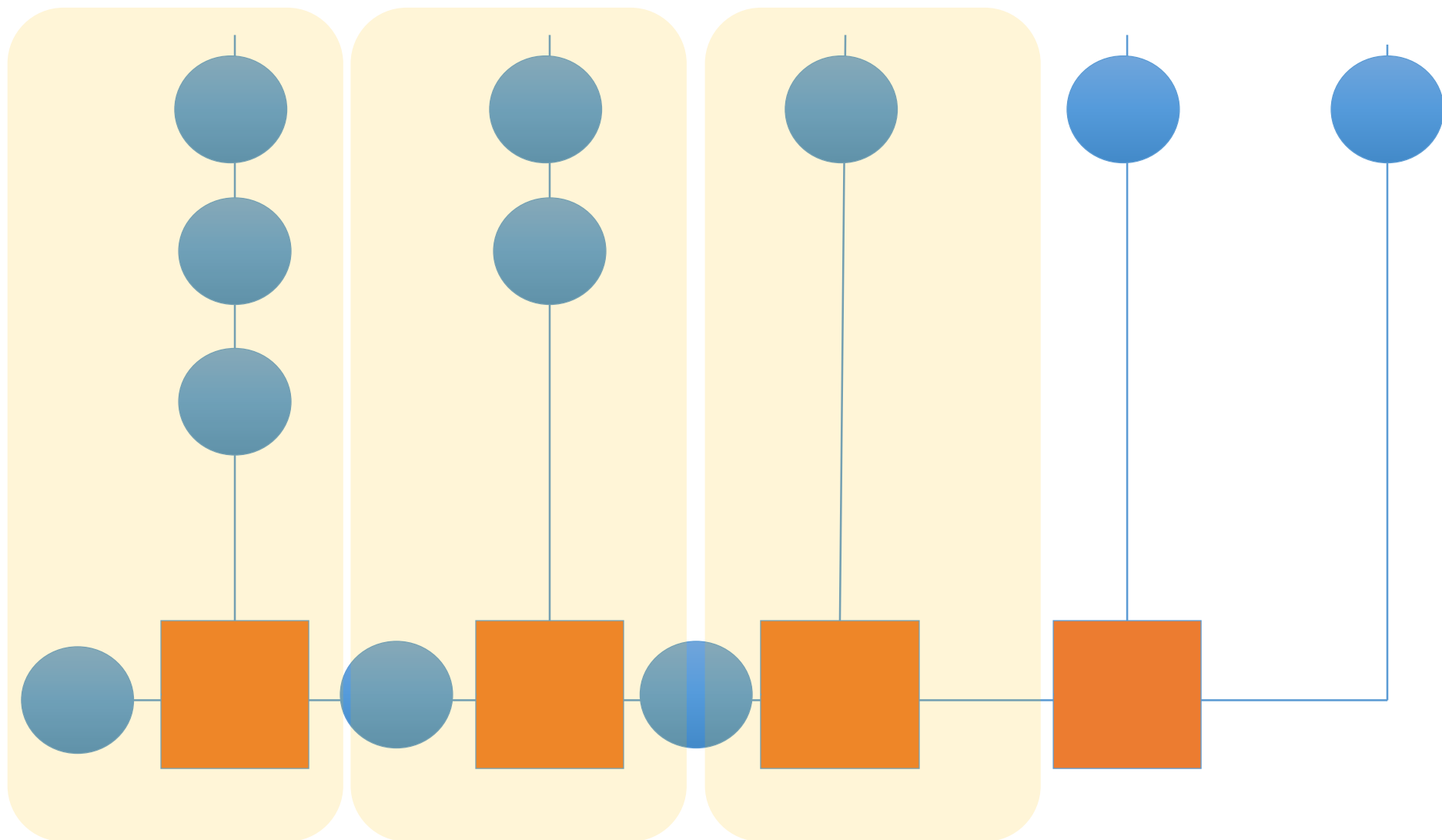


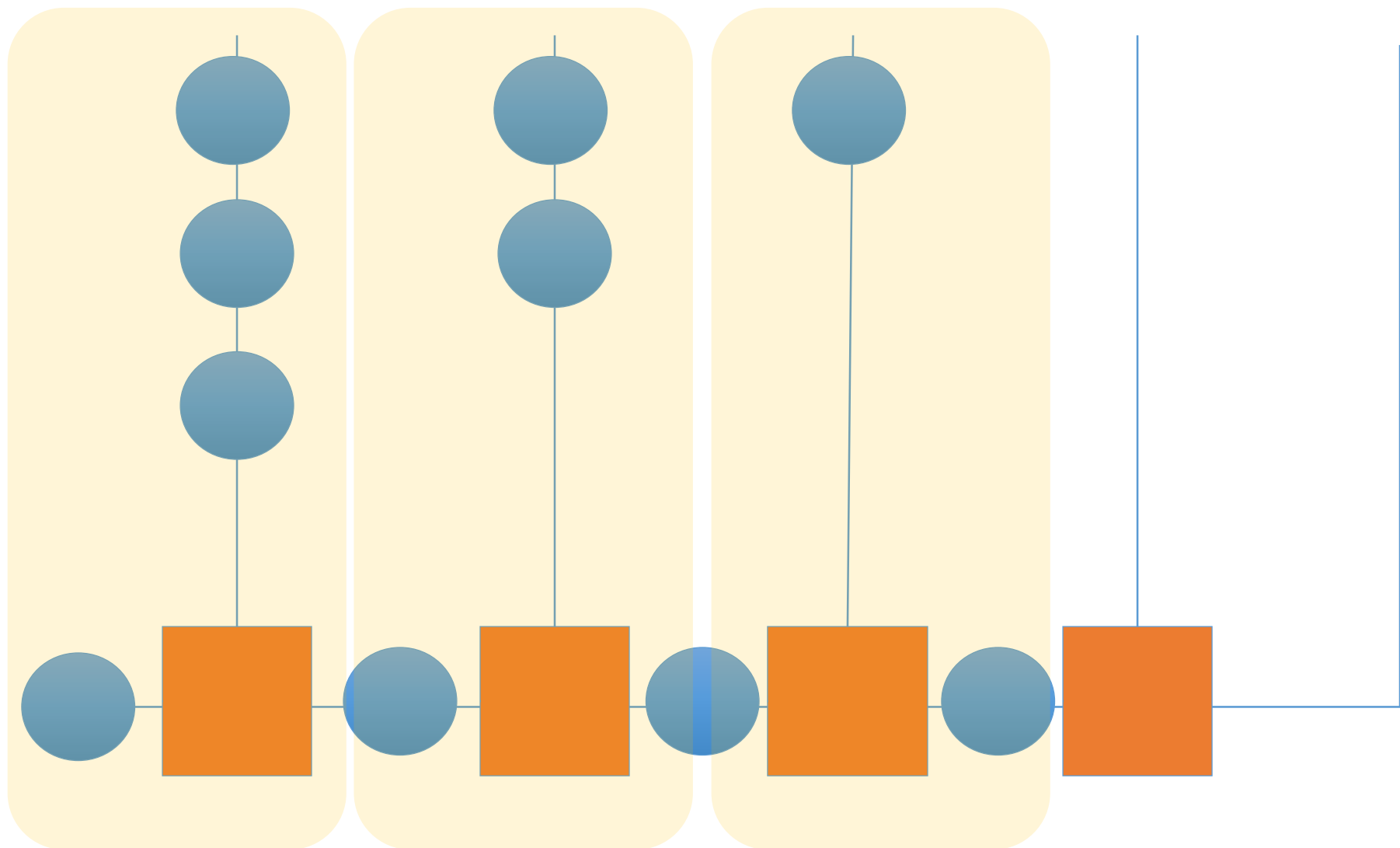












Users!

Plessey video motion estimation

“Using muFP, the array processing element was described in just one line of code and the complete array required four lines of muFP description. muFP enabled the effects of adding or moving data latches within the array to be assessed quickly.”

Bhandal et al, An array processor for video picture motion estimation,
Systolic Array Processors, 1990, Prentice Hall

work with Plessey done by G. Jones and W. Luk

Lava

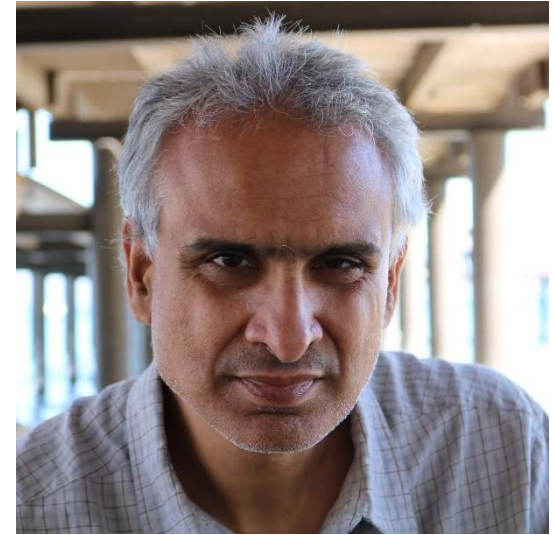
Semantics

muFP

+

Functional
Geometry

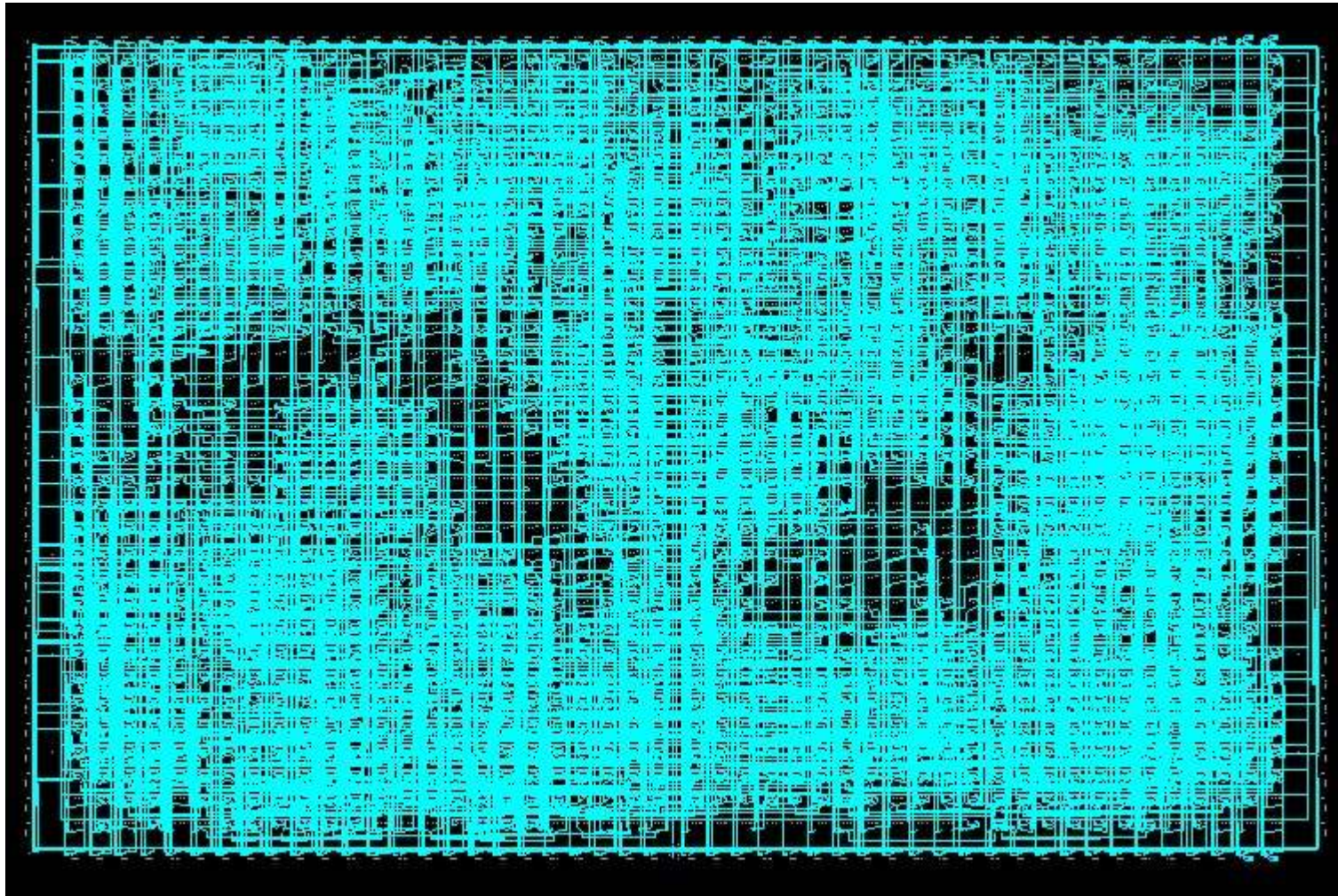
Placement



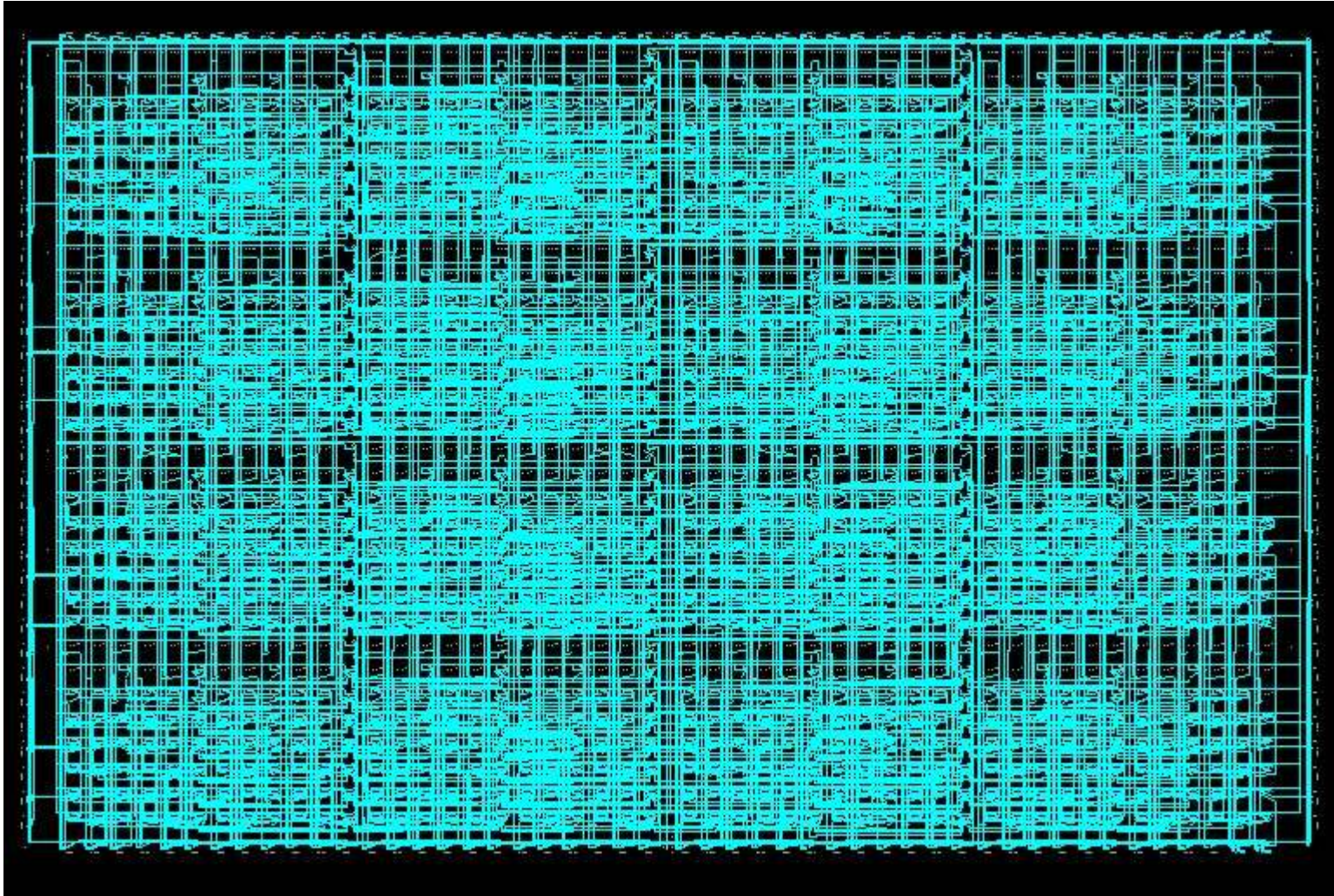
Satnam Singh
Xilinx

➔ FPGA layouts on Xilinx chips

Four adder trees—no placement



Four adder trees—Lava



Intel

$$4195835.0 - 3145727.0 * (4195835.0 / 3145727.0) = 0$$

Intel

$$4195835.0 - 3145727.0 * (4195835.0 / 3145727.0) = 0$$

Flawed Pentium

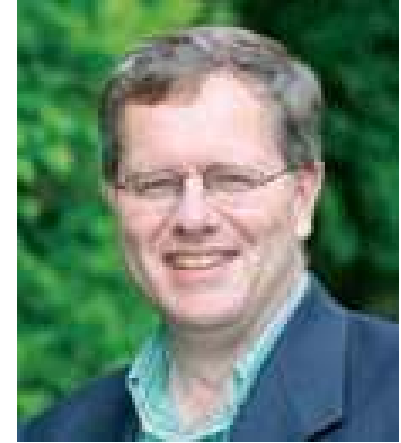
$$4195835.0 - 3145727.0 * (4195835.0 / 3145727.0) = 256$$

\$475 million



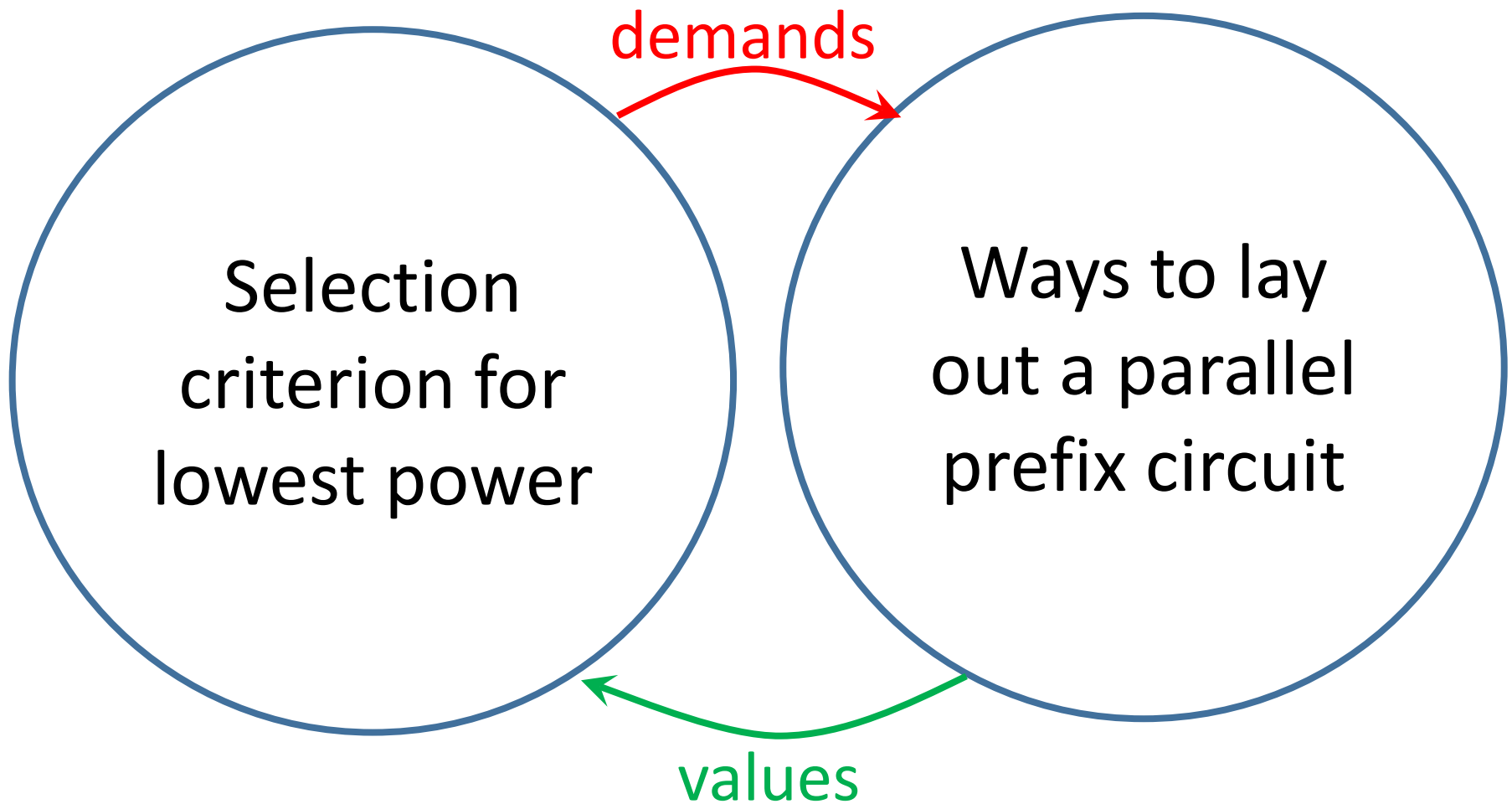
fl

Lazy functional language,
1000s users

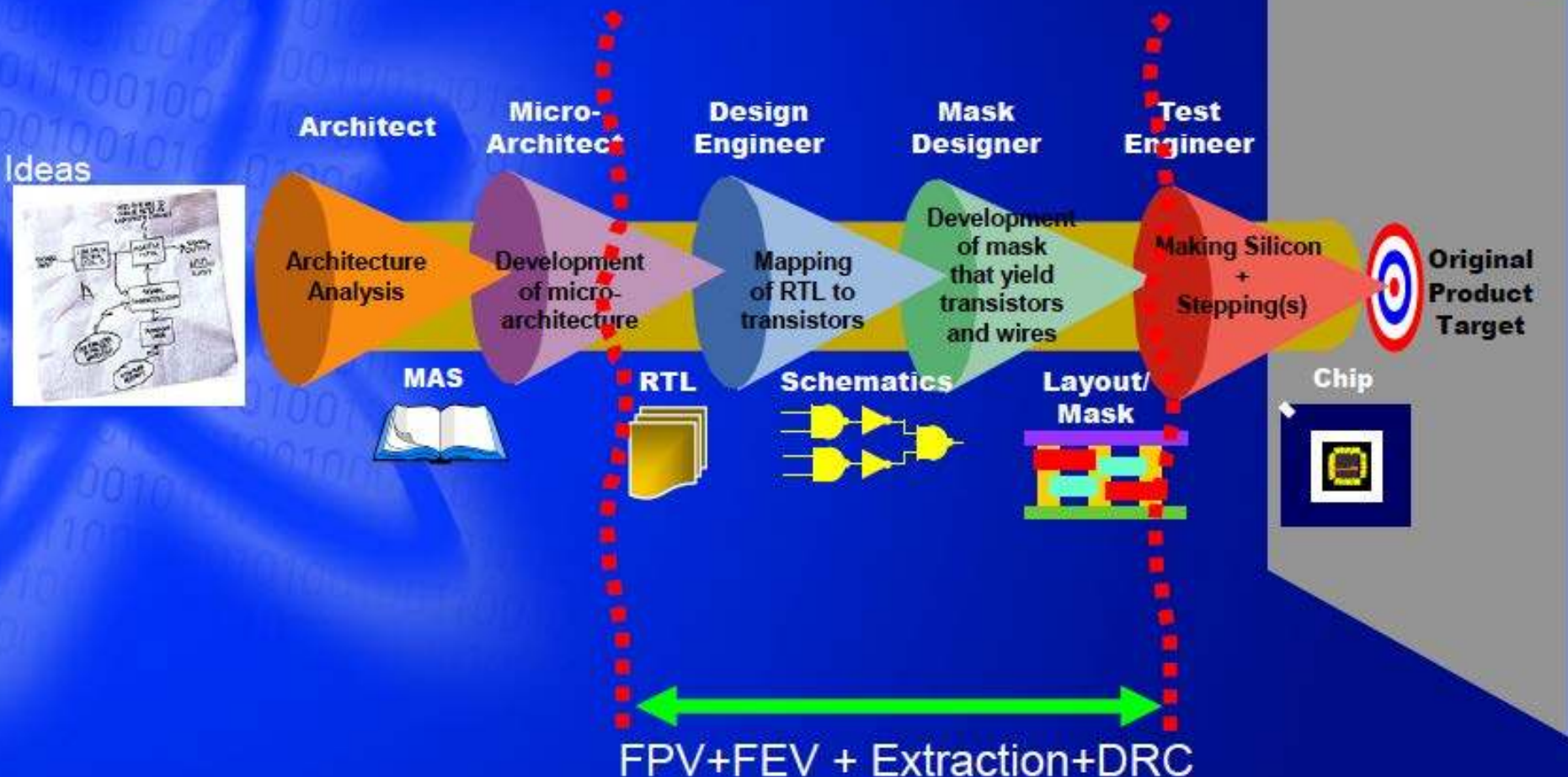


- Design
- High-level specification
- Scripting
- Implementation of formal verification tools and theorem provers
- Object language for theorem proving

Thanks to Carl Seger (formerly of Intel)



Solid Formal Link with Good Return of the Investment

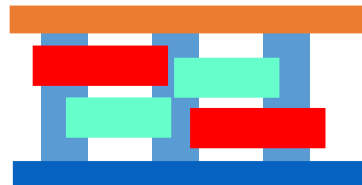


Thanks to Carl Seger (formerly of Intel)

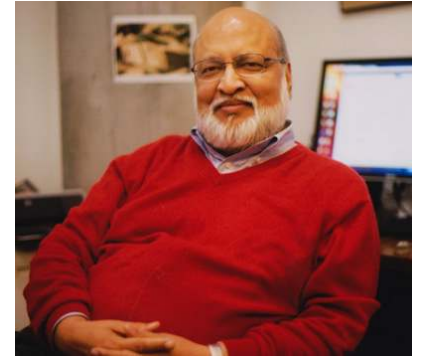
SPEC



**Layout/
Mask**



Bluespec—FP for hardware



Haskell-like language (architecture) +
atomic transition rules (H/W modelling)

Frees designers to explore *better algorithms*,
making major architectural change easy



Types, Functional Programming and
Atomic Transactions in Hardware Design
Rishiyur Nikhil LNCS 8000

Bluecheck

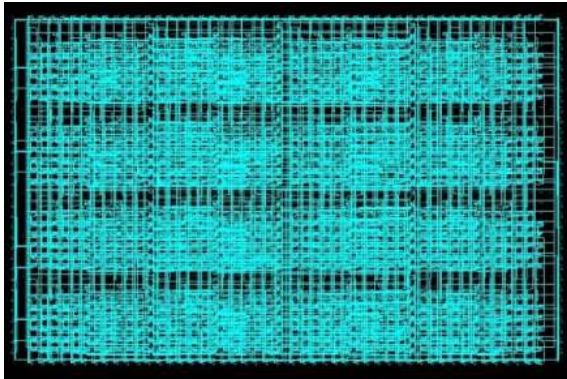
- QuickCheck in Bluespec!
- Generates and shrinks tests *on the FPGA!*

[A Generic Synthesisable Test Bench \(Naylor and Moore, Memocode 2015\)](#)

two $f\ x = f\ (f\ x)$

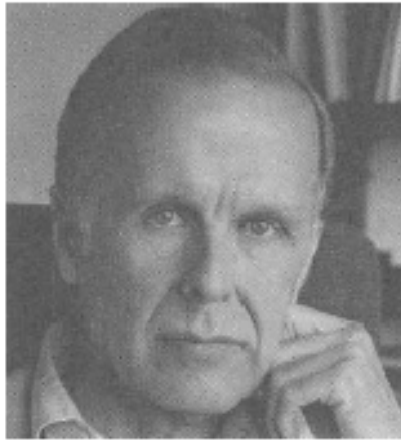
one $f\ x = f\ x$

zero $f\ x = x$



**Whole
values**

**Combining
forms**



**Simple
laws**

**Functions as
representations**